计算机网络

# 第七章 应用层

授课教师：张新有

Email：Ben_eric@qq.com

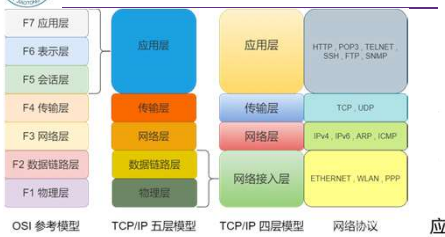
计算机与人工智能学院
School of Computing and Artificial Intelligence

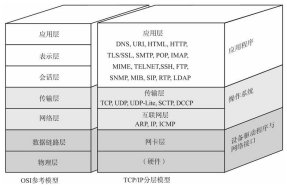
## 本章目标

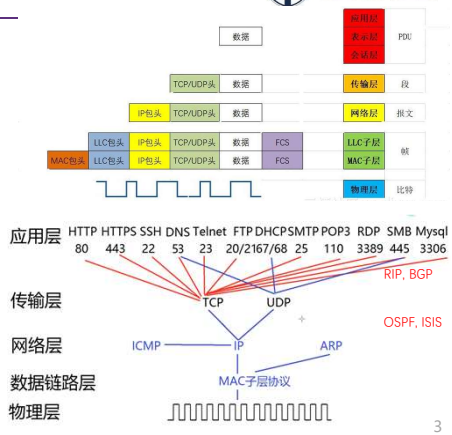
1. 掌握应用进程通信方式以及服务进程工作模式
2. 掌握域名系统DNS基本原理和工作机制
3. 掌握电子邮件系统email体系结构及基本工作原理
4. 了解WWW系统结构框架、静态和动态Web及其应用技术，掌握HTTP协议及其工作原理
5. 掌握Telnet、FTP、SNMP等应用层协议的工作原理
6. 了解流媒体基本概念、数字音视频与编码、流式存储媒体、流媒体动态自适应传输
7. 了解内容分发背景及内容分发网络CDN、及P2P网络工作机制



## 网络体系结构








## 本章内容

- 7.1应用进程和应用层（p3）
- 7.2 域名系统（p21）
- 7.3 电子邮件（p70）
- 7.4 WWW（p102）
- 7.5 其它应用层协议（p147）
- 7.6 流式音频和视频
- 7.7 内容分发网络

1. 应用进程通信方式：C/S;B/S;P2P
2. 服务器进程工作方式：循环 vs 并发



## 应用层协议和应用进程

- 应用层协议都是为了解决某一应用问题，由位于不同主机中的多个应用进程之间的通信和协同工作应遵守的规则
  - 两台主机通信实际是其上对应的两个应用进程(process)在通信
  - 应用进程: 为解决具体应用问题而彼此通信的进程
- 应用层协议的具体内容就是规定应用进程在通信时所遵循的协议；
- 网络进程间的关系：
  - 客户/服务器（C/S, Client/Server）方式
  - 浏览器/服务器（B/S，Browser/Server） 方式
  - 对等（P2P，Peer to Peer）方式



## 1. 应用进程通信方式

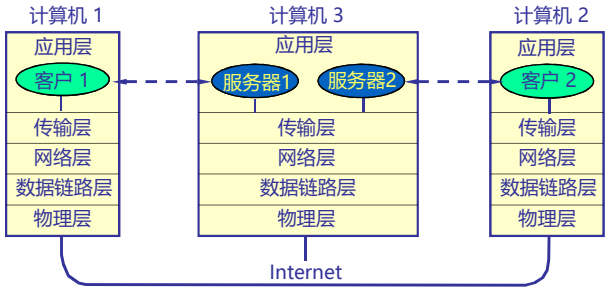
### (1) 客户/服务器（C/S, Client/Server）方式

- 应用层的许多协议是基于C/S方式，例如，在移动互联网环境下，每个应用APP都是一个客户端
  - 客户(client)和服务器(server)是指通信中所涉及的2个应用进程
  - 客户/服务器方式描述的是应用进程之间服务和被服务的关系
  - 客户是服务请求方（主动请求服务，被服务）
  - 服务器是服务提供方（被动接受服务请求，提供服务）
- C/S方式可以是面向连接的，也可以是无连接的
- 面向连接时，C/S通信关系一旦建立，通信就是双向的，双方地位平等，都可发送和接收数据



## 应用进程通信方式

- 功能较强的计算机可同时运行多个服务器进程






## 应用进程通信方式

- 客户进程client的特点
  - 在进行通信时临时成为客户，它也可在本地进行其它的相关计算
  - 用户计算机上运行，在打算通信时主动向远地服务器发起通信
  - 客户方必须知道服务器进程所在主机的IP地址和端口号才能发出服务请求
  - 客户可以与多个服务器进行通信

## 应用进程通信方式

- 服务器进程Server的特点
  - 专门用来提供某种服务的程序，可“同时”处理多个远地或本地客户的请求
  - 必须始终处于运行状态才有可能提供服务
  - 通信开始之前服务器进程不需要知道客户进程所在主机的IP地址，无论客户请求来自哪里，服务器进程被动等待服务请求的到来即可
  - 通常是当系统启动时即自动调用并一直运行着。某些服务器程序也可以由用户或其它进程在通信前启动
  - 被动等待并接受来自多个客户的通信请求（listen→accept）



## 应用进程通信方式

(2) 浏览器/服务器(B/S，Browser/Server) 方式

- B/S方式可以看做C/S方式的特例，即客户软件改为浏览器了
- B/S方式采取浏览器请求、服务器响应的工作模式
- 在B/S方式下，用户界面完全通过浏览器实现，一部分事务逻辑在前端实现，但主要的事务逻辑在服务器端实现

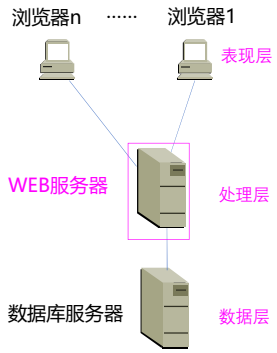
以HTTP为例：





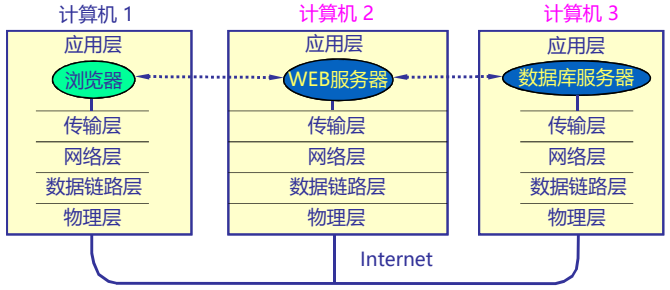

## 应用进程通信方式

- B/S方式通常采取3层架构实现
  - 数据层：由数据库服务器承担数据处理逻辑，其任务是接受Web服务器对数据库服务器提出的数据操作请求，然后由数据库服务器进行数据处理，并把处理结果返回给web服务器
  - 处理层：由Web服务器承担业务处理逻辑和页面存储管理，接受客户浏览器任务请求，执行相应事务处理
  - 表现层：浏览器承担网页信息的浏览功能，以超文本格式实现信息的输入和浏览
- 实际部署时，也可以把数据库服务器和web服务器部署在同一台设备上






## 应用进程通信方式




B/S方式的3层架构示意

## 应用进程通信方式

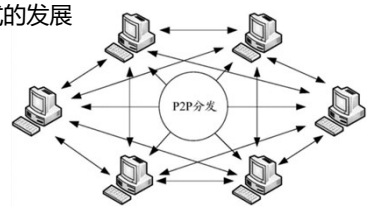
- B/S方式的特点
  - 界面统一，使用简单。客户端只需要安装浏览器软件
  - 易于维护。对应用系统升级时，只需更新服务器端的软件，减轻了系统维护和升级的成本
  - 可扩展性好。采用标准的TCP/IP和HTTP协议等，具有良好扩展性
  - 信息共享度高。HTML是数据格式的一个开放标准，目前大多数流行的软件均支持HTML，XHTML
  - 需要注意的是，在一种浏览器环境下开发的界面在另一种浏览器环境下可能有不完全适配的情况，这时需要安装对应的浏览器。



## 应用进程通信方式

(3) 对等（P2P，Peer to Peer）方式

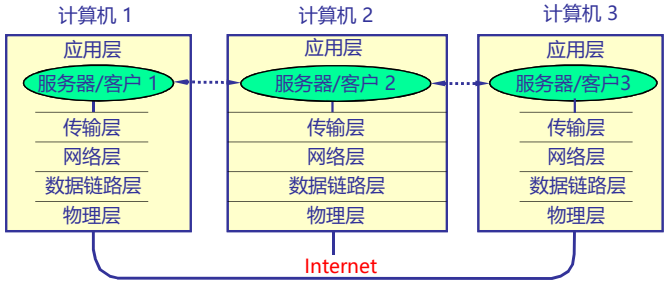
- 对等方式是指两个进程在通信时并不区分服务的请求方和服务的提供方
  - 只要两个主机都运行P2P软件，它们就可以进行平等、对等的通信
  - 双方都可以下载对方存储在硬盘中的共享文档，如果权限允许的话
- 音频/视频应用推动了P2P对等通信方式的发展
- 音频/视频流量中P2P已占主要比例






## 应用进程通信方式

- P2P方式从本质上看仍然是使用了C/S方式，但强调的是通信过程中的对等，这时每一个P2P进程既是客户同时也是服务器






## 2. 服务器进程工作方式

- 循环方式(iterative mode)迭代方式
  - 一次只运行一个服务进程
  - 当有多个客户进程请求服务时，服务进程就按请求的先后顺序依次做出响应 (阻塞方式)
- 并发方式(concurrent mode)
  - 可以同时运行多个服务进程
  - 每一个服务进程都对某个特定的客户进程做出响应 (非阻塞方式)

## 服务器进程工作方式

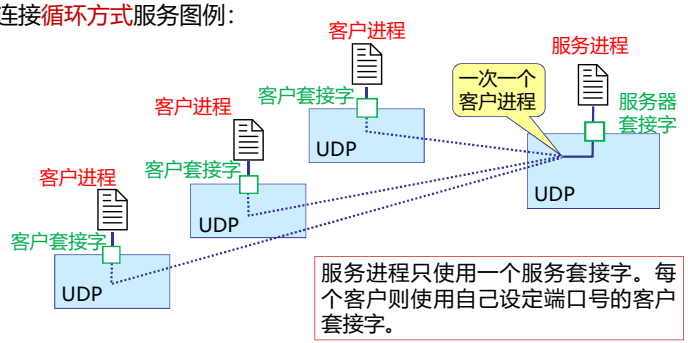
(1) 无连接的循环方式服务

- 使用无连接的UDP服务进程通常都工作在循环方式。即一个服务进程在同一时间只能向一个客户进程提供服务。(顺序服务)
  - 服务进程收到客户进程的请求后，就发送UDP用户数据报响应该客户
  - 对其他客户进程发来的请求则暂时不予理睬，这些请求都在服务端的队列中排队等候服务进程的处理
  - 当服务进程处理完毕一个请求时，就从队列中读取来自下一个客户进程的请求，然后继续处理



## 服务器进程工作方式

无连接循环方式服务图例：






## 服务器进程工作方式

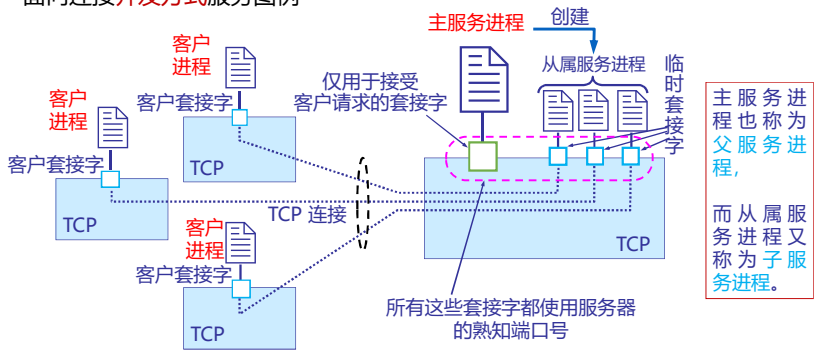
(2) 面向连接的并发方式服务

- 面向连接的TCP服务进程通常都工作在并发服务方式，服务进程在同一时间可同时向多个客户进程提供服务。(并发服务)
- 在TCP服务进程与多个客户进程之间必须建立多条TCP连接，每条TCP连接在其数据传送完毕后释放
- 一个TCP连接对应一连服务端口
- 主服务进程在熟知端口等待客户发出的请求。一旦收到客户请求，就创建一个从属服务进程，并指明从属服务进程使用链接套接字与该客户建立TCP连接，主服务进程继续在原熟知端口等待，向其他客户提供服务



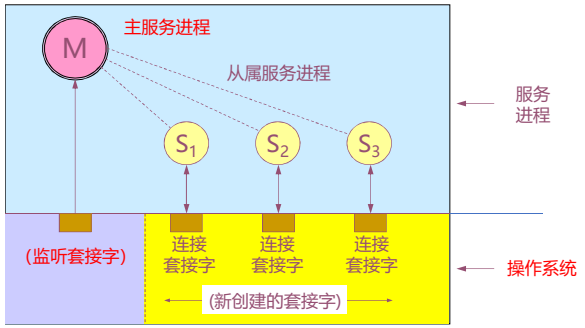
## 服务器进程工作方式

面向连接并发方式服务图例



# 服务器进程工作方式

并发方式工作的服务进程

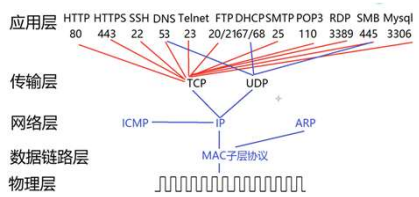





# 本章内容

7.1 应用层概述（p3）
7.2 域名系统DNS（p21）
7.3 电子邮件email（p70）
7.4 WWW（p102）
7.5 其它应用层协议（p47）
7.6 流式音频和视频
7.7 内容分发

1. 历史和概述
2. 域名系统名字空间和层次结构
3. 域名服务器
4. 域名解析过程
5. 域名系统查询和响应(选)
6. 域名系统高速缓存
7. 域名系统隐私(选)






# 1. 历史与概述

## （1）域名系统历史

- ARPANET时期
  - Hosts.txt文件列出了所有计算机名称和它们的IP地址
  - 所有主机在晚上从指定站点上获取Hosts.txt文件
  - Hosts.txt文件的使用在当时表现不错
- 互联网发展
  - Hosts.txt文件变得越来越大
  - 需要集中管理来防止主机名冲突
- 1987年发布了域名系统DNS的RFC文档（RFC1034/1035）
  - 采用了一种层次的、基于域的命名模式，并使用分布式数据库系统实现
  - 主要用途是将主机名映射成IP地址





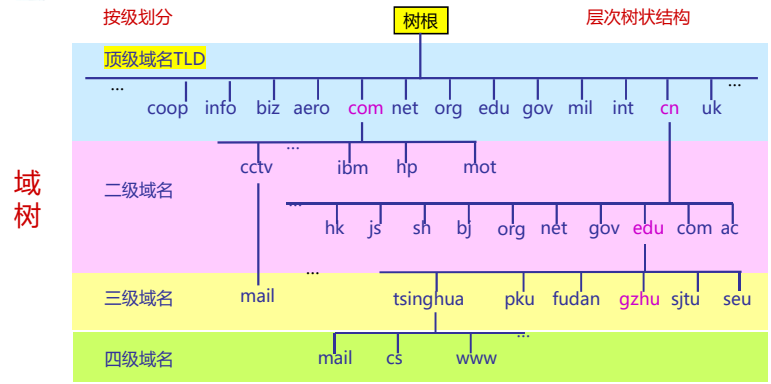

# 历史与概述

## （2）域名系统概述

- 域名系统（DNS，Domain Name System）是互联网重要的基础设施之一，向所有需要域名解析的应用提供服务，负责将可读性好的域名映射成IP地址
- Internet采用层次结构的命名树作为主机的名字，并使用分布式的域名系统DNS。Internet的DNS是一个联机分布式数据库系统
- 名字到IP的解析是由若干个域名服务器程序协同完成的。域名服务器程序在专设的结点上运行，相应的结点也称为名字服务器(Name Server)或域名服务器(Domain Name Server)



## 2. DNS名字空间和层次结构






## DNS名字空间和层次结构

### Internet的域名结构

- Internet的域名结构采用了层次树状结构的命名方法
- 域名的结构由若干个分量组成，各分量之间用小数点(.)隔开，总长不超过255个字符
- 各分量分别代表不同级别的域名(≤63字符)
- 合法域名中，点“.”的个数至少为一个
- 通常，点“.”对应的英文单词为dot，也可以读为point

…. 三级域名 . 二级域名 . 顶级域名；如4级域名www.swjtu.edu.cn



## DNS名字空间和层次结构

### 1) 顶级域名TLD（Top Level Domain）一般有三类

- 国家或地区顶级域nTLD，也记为ccTLD (cc: country code)
  - 例如.cn 表示中国，.us 表示美国，.uk 表示英国。目前有300多个
- 网络基础设施域.arpa (Address and Routing Parameter Area)
  - 专用于Internet基础设施目的
  - 目前有二级域ipv6.arpa；iris.arpa；in-addr.arpa；uri.arpa；urn.arpa；home.arpa；in-addr-servers.arpa；ipv4only.arpa等
- 通用顶级域gTLD
  - 早期规定了20个通用顶级域名，2011年批准新通用顶级域名(New generic Top-level Domain，New gTLD)
  - 截至2020年，已注册有1200多个通用顶级域名



## DNS名字空间和层次结构

### 早期的通用顶级域名

- .com 表示公司企业 (commerce)
- .net 表示网络服务机构，例如NIC和NOC (network)
- .org 表示非赢利性组织 (organization)
- .edu 表示教育机构(美国专用) (education)
- .gov 表示政府部门(美国专用) (government)
- .mil 表示军事部门(美国专用) (military)
- .int表示政府间国际合约建立的国际性组织 (international)
- .mobi 用于提供移动产品和服务的用户和供应商 (mobile)

## DNS名字空间和层次结构

早期的通用顶级域名

- .aero 用于航空运输企业
- .biz 用于商业 (business)
- .cat 用于(西班牙)加泰罗尼亚语言和文化团体
- .coop 用于合作团体
- .info 适用于各种情况
- .jobs 用于人力资源管理者



## DNS名字空间和层次结构

早期的通用顶级域名

- .museum 用于博物馆
- .name 用于个人
- .pro 用于有资质的专业人员及实体，如会计、律师和医师等自由职业者
- .tel 用于商业和个人公布其联系方式
- .travel 用于旅游业实体
- 2012年开始，公司名可以作为新的顶级域名(18万美元)
- 也可以注册 .中国、.公司、.网络 等顶级中文域名(多语种域名国际标准 RFC3454、RFC3490、RFC3491、RFC3492)



## DNS名字空间和层次结构

2) 二级域名

国家顶级域名 .cn下的二级域名分为三类

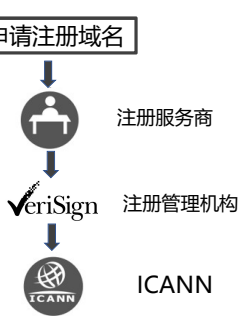
- 类别域名7个
  - .edu.cn 教育
  - .gov.cn 政府
  - .org.cn 非营利组织
  - .net.cn 网络服务
  - .com.cn 工商金融等企业
  - .ac.cn 科研
  - .mil.cn 国防机构
- 行政区域名34个：省、直辖市、自治区、特区等行政区域名，每个行政区域名为两个字母，例如北京bj、河北he等。.bj.cn；.sc.cn
- 无类别域名：例如 www.google.cn、www.tianya.cn等



## DNS名字空间和层次结构

(3) 域名的管理

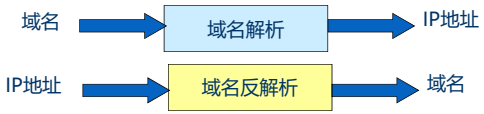
- 域名管理机构分级负责域名注册
  - Internet的域名管理机构： ICANN (Internet Corporation for Assigned Names and Numbers) www.icann.org
  - ccTLD下的二级域名该国自行确定
  - 三级域名注册由其所属二级域名机构负责，以此类推
- .edu.cn下三级域名注册由CERNET负责
- 我国其它二级域名注册由中国互联网络信息中心(CNNIC)负责




注册管理机构与注册服务商之间关系

## 3. 域名服务器

- 保存关于域树(domain tree)的结构和设置信息的服务器程序称为名字服务器(name server)或域名服务器，负责域名解析工作；
- 每个域名服务器具有指向其它有关域名服务器的信息，当需要查询其它域的信息时，能够知道如何找到其他名字服务器，使域名解析过程能够完成；
- 域名与IP地址可以是一对一、一对多或者多对一的关系
- 域名解析过程对用户透明






## 域名服务器

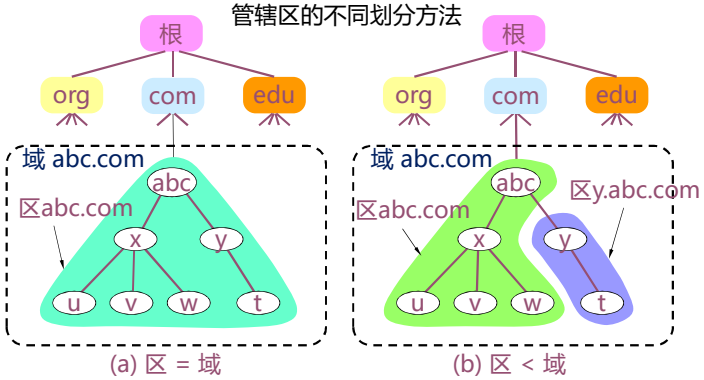
（1）域名系统的区域

- Internet上名字服务器系统也按域名层次树状安排。每个名字服务器管辖一部分域
- 一个名字服务器所负责或管辖（有权限的）范围称为管辖区(zone)，简称为区，区和域是相对的。
- 各单位根据具体情况来划分自己管辖区。但在一个区中的所有节点必须是能够连通的
- 名字服务器的管辖范围以“区zone”为单位，而不是以“域”为单位
- 管辖区是域名“域”的子集。管辖区可以小于或等于域，但不可能大于域



## 域名服务器

管辖区的不同划分方法






## 域名服务器

（2）名字服务器类别

- 总体上，域名系统的名字服务器分为两大类
  - 权威名字服务器(authoritative name server)
    - 一种根据本地知识知道一个DNS区(zone)的内容的服务器，它可以回答有关该DNS区的查询而无需查询其他服务器
    - 每个DNS区至少应有一个IPv4可访问的权威名字服务器提供服务
  - 递归解析器(recursive resolver)
    - 以递归方式运行的、使用户程序联系域名字服务器的程序。

## 域名服务器

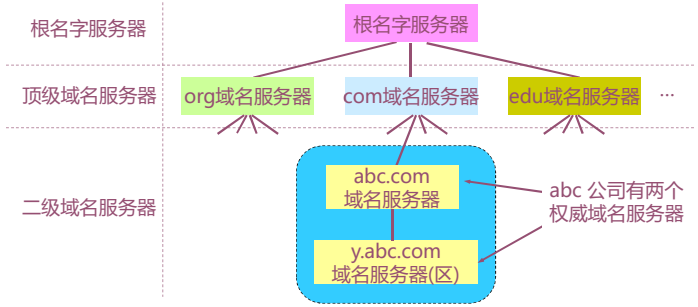
权威名字服务器类别

- 根据对应域的层次，权威名字服务器又进一步分为以下类别
  - 根名字服务器(root name server) /根服务器(root server)
  - 顶级域名字服务器(TLD name server)
  - 二级域名字服务器(second level domain name server)
  - 三级域名字服务器(third level domain name server)
  - .....
- 三级域及以下的名字服务器(如gzhu.edu.cn)通常在用户本地区域，因此三级域及以下的名字服务也统称为本地域名服务器



## 域名服务器

层次树状结构的权威名字服务器






## 域名服务器

1) 根服务器

- 根服务器是最高层次的名字服务器，是根服务器系统云的入口点的名称。在DNS技术社区内，根服务器是特定的任意播(anycast)实例，即回答关于根区域内容查询的权威名字服务器
- 每个根服务器都知道所有的顶级域名服务器的域名及其IP地址
- 路由器把查询转发到距离递归服务器最近的一个根服务器(采用了任意播技术)，提高了查询效率
- 根服务器并不直接把主机用户所查的域名转换成IP地址



## 域名服务器

1) 根服务器

- 根服务器共有13套(不是13台机器)，这些根服务器相应的域名分别是：a.rootservers.net ~ m.rootservers.net
- 更改根服务器数据只在a.rootservers.net上进行，然后同步到另外12套中，这样既能保证数据一致性，也提高了域名服务可靠性
- 每套都可以有多个镜像(mirrored)根服务器，其内容定期与上述对应的根服务器同步。注意，同步需要一定的时间才能完成
- 根服务器之间采用任意播(anycast)技术互联。目前全球已设置了1000多台镜像根服务器，用户域名解析请求时延进一步降低。这样可以方便用户，使世界上大部分DNS域名服务器都能就近找到一个根服务器

## 域名服务器

### 2) 顶级域名字服务器

- 顶级域(TLD)名字服务器负责管理在该顶级域名服务器注册的所有二级域名
- 当收到DNS查询请求时就给出相应的回答（可能是最后的结果，也可能是下一步应当找的二级域名字服务器的 IP 地址）



## 域名服务器

### 3) 二级域名字服务器

- 每一个主机都必须在某个二级域名字服务器处注册登记。因此二级域名字服务器知道其管辖的主机名应当转换成什么IP地址
  - 各个单位根据自己的具体情况把本单位的域名划为若干个域名管辖区(zone)，也可简称为区
- 每个区设置相应的权威名字服务器，用来保存该区中的所有主机的域名到IP地址的映射
- 一般来说，每个名字服务器只负责解析本辖域(区)的域名



## 域名服务器

### 4) 递归/本地域名服务器

- 每一个Internet服务提供者ISP(Internet Service Provider), 都至少有一个递归（本地）域名服务器，距离用户主机较近
- 当用户主机发出DNS查询报文时，这个查询报文就首先被送往该主机所在区域的本地服务器
- 如果所要查询的主机也处在本地ISP的管辖范围，则本地域名服务器就能进行域名解析，否则就需要再询问其他的域名服务器
- 不同网络中的本地域名服务器的部署层次可能不同
- 最简单的部署层次是只有一层本地域名字服务器，即三级域名字服务器



## 域名服务器

### 4) 递归/本地服务器

- 实际部署中，解析请求路径上的递归/本地域名服务器可能有多层
- 在递归/本地域名服务器有多层的结构中，当本层服务器没有解析结果时，通常是逐级向上递归进行查询
- 当查询请求到达本地域最上一层名字服务器时，该层服务器对自己无法解析的任何一个域名，首先求助于根服务器（而不是二级域名字服务器），开始逐步迭代查询
- 在谷歌公司的公共DNS服务系统中，把根区文件信息也同步到它的递归解析器上，这时就不再需要传统的查询根服务器的过程，从而提高查询效率

# 4. 域名解析过程

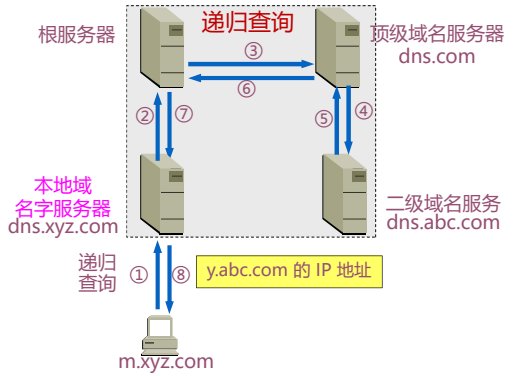
域名解析过程概述

- 当某一应用进程需要进行域名解析时，该应用进程将域名放在DNS请求报文（UDP数据报，port为53）发给递归/本地服务器（使用UDP是为了减少开销）。递归服务器得到查询结果后，将对应IP地址放在应答报文中返回给应用进程（注：DNS不仅使用了UDP协议，也使用了TCP协议）
- 域名查询有递归查询(recursive query)和迭代查询(或循环查询，iterative query)两种方式
  - 主机向递归解析器/本地域名字服务器的查询一般采用递归查询
  - 递归解析器/本地域名字服务器向根服务器可以采用递归查询，但一般优先采用迭代查询



# 域名解析过程

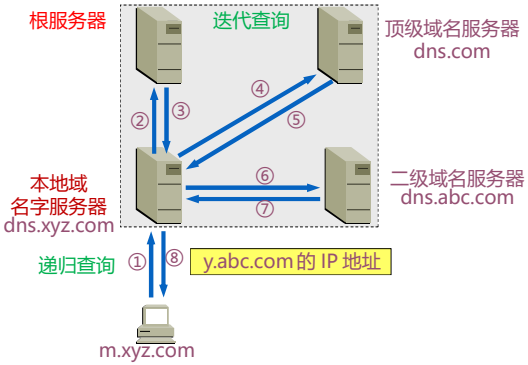
- 递归查询
  - 当收到查询请求报文的域名服务器不知被查询域名的IP地址时，该域名服务器就以DNS客户的身份向下一步应查询的域名服务器发出查询请求，即替递归服务器继续查询。
- 较少使用






# 域名解析过程

- 迭代查询
  - 当收到查询请求报文的域名服务器不知道被查询域名的IP地址时，就把自己知道的下一步应查询的域名服务器IP地址告诉本地域名字服务器，由本地域名字服务器继续向该域名服务器查询，直到得到所要解析的域名的IP地址，或者查询不到所要解析的域名的IP地址
- 通常使用






# 5. 域名系统查询和响应

常用的资源(Resource)类型type

| 类型代码 | 描述 | 功能 |
|---|---|---|
| SOA (Start Of Authority) | 权威记录的起始 | 指定有关DNS区域的权威性信息，包含主要名字服务器、域名管理员的电邮地址、域名的流水式编号、和几个有关刷新区域的定时器 |
| A (Address) | IP地址记录 | 传回一个 32 比特的 IPv4 地址 |
| AAAA (Address) | IPv6地址记录 | 传回一个 128 比特的 IPv6 地址 |
| MX (Mail Exchanger) | 电子邮件交互记录 | 引导域名到该域名的邮件传输代理（MTA, Message Transfer Agents）列表 |
| CNAME (Canonical NAME) | 规范名称记录 | 主机名字的别名：域名系统将会继续尝试查找新的名字 |

# 域名系统查询和响应

常用的资源(Resource)类型type

| 类型代码 | 描述 | 功能 |
|---|---|---|
| PTR(pointer) | 指针记录 | 引导至一个规范名称(Canonical Name)，最常用来运行反向 DNS 查找 |
| NS (Name Server) | 名字服务器记录 | 用于说明这个区域有哪些DNS服务器负责解析 |
| SPF (sender policy framework) | SPF记录 | 让一个域把邮件服务信息进行编码，即域内哪台机器负责把邮件发送到Internet其余地方，有助于接受机器检查邮件是否有效 |
| SRV (Service) | 服务定位器 | 把主机标识为域内的一种给定服务。例如cs.washington.edu的web服务器可以标识成cockatoo.cs.washington.edu |
| TXT (Text) | 文本记录 | 最初用于允许域以任意方式标识自己，如今通常被编码成机器可读信息，一般是SPF信息 |



# 域名系统查询和响应

DNS报文格式

- 分为三部分：基础结构(报文首部)、问题、资源记录(RR, Resource Record)
- 报文类型分为查询请求(query)和查询响应(reply)两类，请求和响应的报文结构基本相同

| | |
|---|---|
| 事务ID（Transaction ID） | 标志（Flags） |
| 问题计数（Questions） | 回答资源记录数（Answer RRs） |
| 权威资源记录计数（Authority RRs） | 附加资源记录数（Additional RRs） |
| 查询问题区域（Queries） | |
| 回答问题区域（Answers） | |
| 权威名称服务器区域（Authoritative name servers） | |
| 附加信息区域（Additional records） | |

基础结构部分

问题部分

资源部分

DNS报文格式



# 域名系统查询和响应

1）DNS报文格式的基础结构部分

- 事务ID：DNS报文的ID标识。对于请求报文和其对应的响应报文，该字段的值是相同的。通过它可以区分 DNS 应答报文是对哪个请求进行响应的
- 标志：DNS报文中的标志字段，具体结构下页说明
- 问题计数：DNS查询请求的数目
- 回答资源记录数：DNS响应的数目
- 权威资源记录数：权威资源记录的数目
- 附加资源记录数：额外的记录数目（权威名字服务器对应IP地址的数目）

DNS报文的基础结构部分

| | |
|---|---|
| 事务ID（Transaction ID） | 标志（Flags） |
| 问题计数（Questions） | 回答资源记录数（Answer RRs） |
| 权威资源记录数（Authority RRs） | 附加资源记录数（Additional RRs） |



# 域名系统查询和响应

DNS报文格式的标志(Flags)字段

- QR（Query/Response）：查询请求/响应的标志信息。查询请求时值为0；响应时值为1
- Opcode：操作码。其中0表示标准查询；1表示反向查询；2表示服务器状态请求
- AA（Authoritative）：授权应答，该字段在响应报文中有效。值为1时表示名称服务器是权威服务器；值为0时表示不是权威服务器
- TC（Truncated）：表示是否被截断。值为1时，表示响应已超过512字节并已被截断，只返回前512个字节

DNS报文的标志字段格式

| QR | Opcode | AA | TC | RD | RA | Z | Rcode |
|---|---|---|---|---|---|---|---|

## 域名系统查询和响应

DNS报文格式的标志(Flags)字段

- RD（Recursion Desired）：期望递归。该字段能在一个查询中设置，并在响应中返回。该标志告诉域名服务器必须处理这个查询，这种方式被称为一个递归查询。如果该位为0，且被请求的域名服务器没有一个授权回答，它将返回一个能解答该查询的其他域名服务器列表。这种方式被称为迭代查询
- RA（Recursion Available）：可用递归。该字段只出现在响应报文中。当值为1时，表示服务器支持递归查询
- Z：保留字段，在所有的请求和响应报文中，它的值必须为0

DNS报文的标志字段格式

| QR | Opcode | AA | TC | RD | RA | Z | Rcode |
|---|---|---|---|---|---|---|---|



## 域名系统查询和响应

DNS报文格式的标志(Flags)字段

- Rcode(Reply code)：返回码字段，表示响应的差错状态。常用Rcode有
  - 当值为0(NoError)时，表示没有错误
  - 当值为1(FormErr)时，表示报文格式错误，服务器不能理解请求的报文
  - 当值为2(ServFail)时，表示域名服务器失败，因为服务器的原因导致没办法处理这个请求
  - 当值为3(NXDomain)时，表示域名不存在，只有对授权域名解析服务器有意义，指出解析的域名不存在
  - 当值为4(NotImp)时，表示查询类型不支持，即域名服务器不支持查询类型
  - 当值为5(Refused)时，表示拒绝应答，一般是服务器由于设置的策略拒绝给出应答，如服务器不希望对某些请求者给出应答

DNS报文的标志字段格式

| QR | Opcode | AA | TC | RD | RA | Z | Rcode |
|---|---|---|---|---|---|---|---|



## 域名系统查询和响应

2）DNS报文格式的问题部分

- 用来显示DNS查询请求的问题，通常只有一个问题
- 包含正在进行的查询信息，包含查询名（被查询主机名字）、查询类型、查询类
  - 查询名(name)：一般为要查询的域名，有时是IP地址，用于反向查询
  - 查询类型(type)：DNS查询请求的资源类型。通常查询类型为A类型，表示由域名获取对应的IP地址
  - 查询类(class)：地址类型，通常为互联网地址，值为1(IN)

问题部分格式

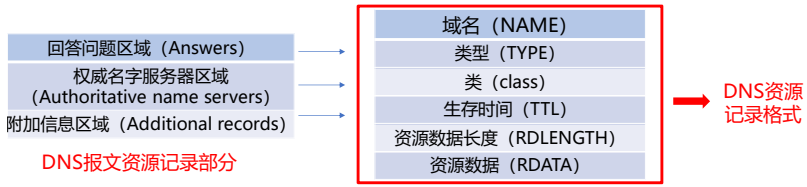
| 查询名 | |
|---|---|
| 查询类型 | 查询类 |



## 域名系统查询和响应

3）DNS报文格式的资源记录(RR, Resource Record)部分

- 资源记录部分是DNS报文格式的最后3个字段，只有在DNS响应报文中才出现，包括回答问题区域字段、权威名字服务器区域字段、附加信息区域字段。这3个字段都采用资源记录的格式



# 域名系统查询和响应

## DNS报文格式的资源记录格式

- 域名：DNS请求的域名
- 类型：资源记录的类型，与问题部分中的查询类型值相同
- 类：地址类型，与问题部分中的查询类值相同
- 生存时间：以秒为单位，表示资源记录的生命周期，一般用于当地址解析程序取出资源记录后决定保存及使用缓存数据的时间。它同时也可明该资源记录的稳定程度，稳定的信息会被分配一个很大的值
- 资源数据长度：资源数据的长度
- 资源数据：表示按查询段要求返回的相关资源记录的数据

DNS资源记录格式

| 域名（NAME） |
| --- |
| 类型（TYPE） |
| 类（class） |
| 生存时间（TTL） |
| 资源数据长度（RDLENGTH） |
| 资源数据（RDATA） |

```
▾ Answers
  ▾ dashboard.snapcraft.io: type A,
      Name: dashboard.snapcraft.io
      Type: A (Host Address) (1)
      Class: IN (0x0001)
      Time to live: 5
      Data length: 4
      Address: 91.189.92.18
```

DNS资源记录抓包示例



# 域名系统查询和响应

## DNS资源记录示例：

RR格式：(name, ttl, class, type, value)

- cs.vu.nl. 86400 IN SOA hostmaster.cs.vu.nl. 2020091500 7200 3600 2419200 7200
- cs.vu.nl. 86400 IN TXT "Vrije Universiteit - Mathematics and Computer Science"
- cs.vu.nl. 7200 IN MX 10 mx2.surfmailfilter.nl.
- cs.vu.nl. 7200 IN MX 10 mx1.surfmailfilter.nl.
- cs.vu.nl. 60 IN A "Vrije Universiteit - Mathematics and Computer Science"
- cs.vu.nl. 86400 IN NS new-ns2.vu.nl.
- google.com. 299 IN AAAA 2a00:1450:4017:804::200e
- 69.174.82.46.in-addr.arpa. 35078 IN PTR p2e52ae45.dip0.t-ipconnect.de.
- p2e52ae45.dip0.t-ipconnect.de. 86400 IN A 46.82.174.69
- www.cs.vu.nl. 60 IN CNAME papac022.vu.nl.



# 域名系统查询和响应

## DNS资源记录的不同类型详解

RR格式：(name, ttl, class, type, value)

- tpye=A
  - name是主机名
  - value是IPv4地址
- tpye=CNAME
  - name是某些“规范”名称的别名，例如www.cs.vu.nl.是主机名papac022.vu.nl.的别名
  - value是IP地址
- tpye=NS
  - name是请求域名
  - value是管理请求域名的权威名称服务器的主机名
- tpye=MX
  - name是请求域名
  - value是与请求域名关联的SMTP邮件服务器的名称



# 域名系统查询和响应

## DNS资源记录的不同类型详解

RR格式：(name, ttl, class, type, value)

- tpye=SOA
  - name是请求域名
  - value是请求域名的DNS区域的权威性信息
- tpye=TXT
  - name是请求域名
  - value是某个主机名或域名设置的说明
- tpye=AAAA
  - name是请求域名
  - value是请求域名的IPv6地址
- tpye=PTR
  - name是IP地址
  - value是与给定 IP 地址关联的域名

## 域名系统查询和响应

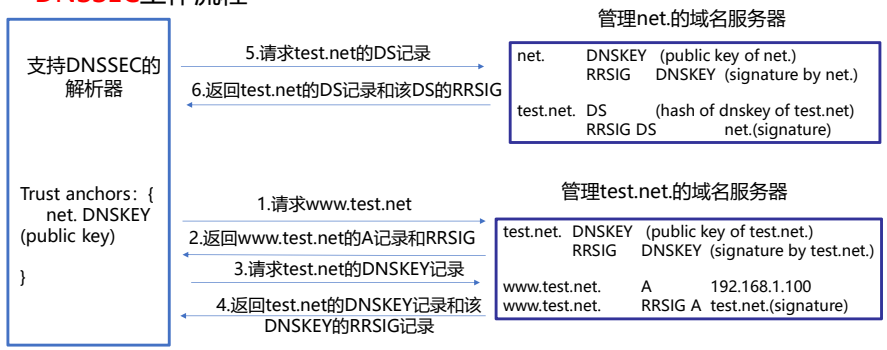
DNSSEC概述

- DNS最初并没有设计强大的安全机制来保证数据的完整性和真实性。多年来，发现了许多威胁DNS系统可靠性和可信性的漏洞
- DNSSEC依靠数字签名保证DNS应答报文的真实性和完整性，包括通过身份验证拒绝DNS数据存在的机制
- 在DNSSEC的使用中，域名服务器用自己的私有密钥对资源记录(Resource Record，RR)进行签名，解析服务器用域名服务器的公开密钥对收到的应答信息进行验证。如果验证失败，表明这一报文可能是假冒的，或者在传输过程、缓存过程中被篡改了；



## 域名系统查询和响应

DNSSEC工作流程






## 域名系统查询和响应

DNSSEC工作流程

- 一个支持DNSSEC的解析器向支持DNSSEC并管理test.net域名的权威域名服务器请求域名www.test.net.
- 解析器除了得到一个标准的A记录(IP 地址)以外，还收到一个同名的RRSIG记录，其中包含test.net这个授权域的数字签名，它是用test.net.的私有密钥来签名的
- 为了验证这一签名的正确性，解析服务器可以再次向test.net.的域名服务器查询响应的公开密钥，即DNSKEY资源记录
- 解析服务器就可以用其中的公钥验证上述记录(关于www.test.net.的RRSIG记录)的真实性与完整性



## 6. 域名系统高速缓存

DNS高速缓存作用

- 为提高DNS查询效率，并减轻根域名服务器的负荷和减少Internet上的DNS查询报文数量，域名服务器广泛使用高速缓存，用来存放最近查询过的域名以及从何处获得域名映射信息的记录
  - ✓ 例如，不久前已有用户查询过域名为y.abc.com的IP地址，则本地域名服务器/递归解析器就不必再向根域名服务器重新查询y.abc.com的IP地址了，而是直接把高速缓存中存放的上次查询结果（即y.abc.com的IP地址）告诉用户

## 域名系统高速缓存

使用高速缓存的好处

- 可以大大减轻根域名服务器的负荷，而且也能够使Internet上的DNS查询请求和回答报文的数量大为减少
  - ✓ 假定本地域名服务器的缓存中并没有 y.abc.com 的IP地址，但存放着顶级域名服务器 dns.com 的IP地址，则本地域名服务器就不必向根域名服务器进行查询，而可以直接向 .com 顶级域名服务器发送查询请求报文
- 域名缓存有多种形式：
  - ✓ 浏览器缓存→系统缓存(hosts文件)→路由器缓存→本地DNS缓存→其他DNS缓存



## 域名系统高速缓存

缓存中保存项目有一定的时限

- 域名到IP地址的绑定可能发生变化（但并不会经常改变）。为保持高速缓存中的内容正确，域名服务器应为每项内容设置计时器并处理超时项目（例如，典型的数值是每个项目只存放48小时）
- 当域名服务器已从缓存中删去某项信息后又被请求查询该项信息，就必须重新查询
- 当权限域名服务器回答一个查询请求时，在响应中都指明绑定有效存在的时间值。增加此时间值可减少网络开销，而减少此时间值可提高域名解析的准确性



## 7. 域名系统隐私

DNS协议的脆弱性

- DNS协议设计之初没有过多考虑安全问题，因此导致DNS存在以下缺陷
  - 几乎所有DNS流量都是基于UDP明文传输的
  - DNS的资源记录未加上任何的认证和加密措施

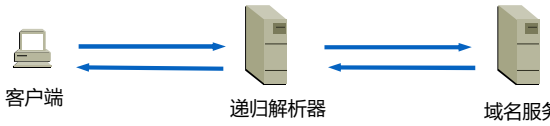
DNS的用户隐私容易被泄漏



## 域名系统隐私

域名解析过程的源IP地址的保密性

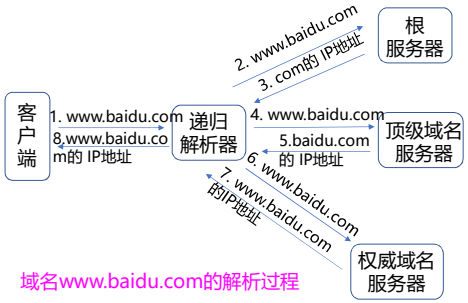
- 源IP地址提供的用户主机的信息，可以看做是用户标识。在一次DNS查询中，不同层面的源IP地址的含义不太一样
  - ✓在递归服务器上，源IP地址基本就是用户主机的IP地址
  - ✓在其他域名服务器上，源IP地址一般是发出查询请求的服务器的IP地址（除非用户在自己主机上部署了本地递归服务）



## 域名系统隐私

域名解析过程的保密性

- QNAME是用户请求的域名全名，它提供了用户的操作信息
- 在域名解析过程中，递归服务器和各层级域名服务器基本都是收到包含QNAME的查询(例如右图中www.baidu.com)
- 不同的QNAME包含的敏感信息不一样。例如一些特定主题网站域名的查询可能会泄露用户的爱好和习惯




域名www.baidu.com的解析过程



## 域名系统隐私

DNS敏感数据主要的泄露途径

- 通信链路窃听：由于DNS的开放性，窃听者可以像窃听其他流量一样监听DNS流量。同时DNS查询并不经过任何的加密手段，因此，任何第三方的机构或个人很容易通过在用户和服务器之间进行搭线窃听，查看到用户所有的DNS查询信息
- 服务器收集：全球现有超过1000万台域名服务器，每天产生的域名查询信息已经达到了万亿级别，同时由于DNS日志已被广泛应用于各种DNS解析器软件中，因此用户的每条查询信息都可以被服务器记录下来
- 域名劫持：将用户的正常域名解析，劫持到特定的网站。



## 本章内容

- 7.1 应用层概述（p3）
- 7.2 域名系统（p21）
- 7.3 电子邮件（p70）
- 7.4 WWW（p102）
- 7.5 其它应用层协议（p147）
- 7.6 流式音频和视频
- 7.7 内容分发网络

1. 体系结构和服务
2. 用户代理
3. 邮件格式
4. 最后传递
5. 邮件交付

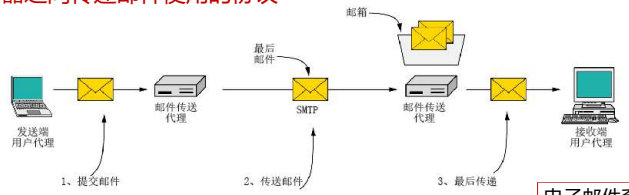


## 电子邮件概述

- 电子邮件（E-mail）是自早期Internet出现以来最广泛的应用，是一种异步通讯媒介
- 优越性
  - 比纸质信件更快更便宜
  - 使用门槛低
  - 收件人可随时上网到自己邮箱读取邮件
  - 电子邮件不仅可传送文字信息，而且还可附上声音和图像等

# 1. 电子邮件系统体系结构

- 用户代理（UA：user agent）——邮件客户端UA
- 传输代理（MTA：message transfer agent）——邮件服务器
- 简单邮件传输协议SMTP（Simple Mail Transfer Protocol）——邮件服务器之间传递邮件使用的协议




电子邮件系统的构成

电子邮件系统采用客户/服务器工作模式



# 电子邮件系统体系结构

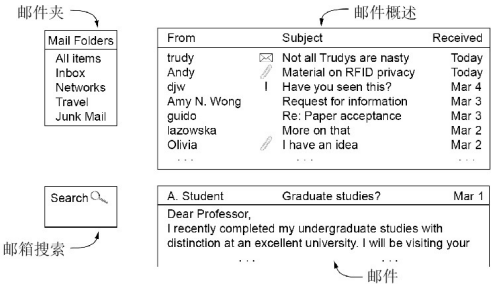
- 用户代理（邮件客户端）
  - 编辑和发送邮件
  - 接收、读取和管理邮件
  - 管理地址簿
  - 无统一标准
- 传输代理（邮件服务器）
  - 邮箱：保存用户收到的邮件
  - 邮件输出队列：存储等待发送的邮件
  - 运行电子邮件协议
- SMTP协议
  - smtp客户: 发送邮件端
  - smtp服务器: 接收邮件端

- 邮箱是邮件服务器中的一块内存区域，其标识即为电子邮件地址（邮箱名）
- 电子邮件地址是一个字符串，用于指定邮件接收者
  - local-part@domain-name
  - domain-name: 域名
  - local-part: 域名中的邮箱名
  - 例：aaa@hotmail.com、david@eyou.com



# 2. 用户代理UA

- 用户代理是一个程序，用户通过它和电子邮件系统交互
- 让用户能够阅读和发送电子邮件（email message）
- 与邮件服务器交互，接收、发送电子邮件
- 常用的用户代理：
- Outlook, foxmail Thunderbird, iPhone mail client，Web客户端等




用户代理接口的典型元素



# 用户代理完成的功能

- 显示入境邮件信息
  - 列表的形式显示入境邮件摘要，摘要信息是每个邮件占一行，并按照某种顺序排列
  - 多数用户代理用多个文件夹保存一个邮箱的入境邮件，并具有自动归档功能
- 邮件处置（message disposition）
  - 删除邮件、转发邮件、保存邮件、过滤/搜索邮件
- 自动处理邮件
  - 邮件过滤、自动回复、自动转发

## 用户代理完成的功能

- 发送邮件
  - 包括创建邮件和回复邮件
  - 发送邮件系统的邮件遵循一个标准格式(请参考后续内容)，最重要的部分是信封，信封中最重要的是目的地址，邮件目的地址必须是邮件传输代理可处理的格式，如abc@xyz.edu.cn
- 邮件列表
  - 允许用户通过一个命令把相同的邮件发送给群发列表中的每个人。可由用户代理在本地维护或远程维护一个在邮件传输代理上的群发名单
- 其它功能
  - 在邮件底部追加签名块（signature block）、更正拼写错误和计算表名该邮件是否有效的数字签名



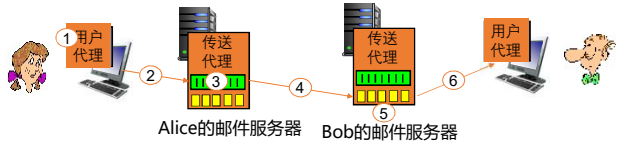
## 3. 邮件传输MTA

- 邮件传输代理将邮件从发件人中继给收件人
- 邮件传输采用的协议是SMTP
  - SMTP利用TCP可靠地从客户向服务器传递邮件，使用端口25
  - 直接投递: 发送端直接到接收端
  - SMTP的3个阶段：连接建立、邮件传送、连接关闭
  - 命令/响应（以HTTP为例）
    - 命令: ASCII字符串
    - 响应: 状态码+短语
  - SMTP是一个简单的ASCII协议，邮件必须为7位ASCII



## 场景: Alice发送邮件给Bob

① Alice使用用户代理撰写发送给 Bob@someschool.edu的邮件
② Alice的用户代理发送邮件到她的邮件服务器；邮件存放在邮件队列
③ SMTP客户端打开与Bob的邮件服务器的TCP连接
④ SMTP客户端通过TCP连接发送Alice的邮件
⑤ Bob的邮件服务器把邮件存放在Bob的邮箱
⑥ Bob调用他的用户代理来读取邮件






## SMTP发送邮件会话实例

- 客户端发送的行用”C:”标识，服务发送的行用”S:”标识
- 来自客户的第一条命令HELO
- RCPT命令可以将一个邮件发送给多个收件人，此处只有一个收件人，只使用一个RCPT命令

```
S: 220 hamburger.edu
C: HELO crepes.fr
S: 250  Hello crepes.fr, pleased to meet you
C: MAIL FROM: <alice@xyz.com>
S: 250 alice@crepes.fr... Sender ok
C: RCPT TO: <bob@someschool.edu>
S: 250 bob@someschool.edu ... Recipient ok
C: DATA
S: 354 Enter mail, end with "." on a line by itself
C: Do you like ketchup?
C: How about pickles?
C: .
S: 250 Message accepted for delivery
C: QUIT
S: 221 hamburger.edu closing connection
```

alice@xyz.com给bob@someschool.edu发送一个邮件

## SMTP 命令（参考）

- HELO <SP> <domain> <CRLF>
- MAIL <SP> FROM:<reverse-path> <CRLF>
- RCPT <SP> TO:<forward-path> <CRLF>
- DATA <CRLF>
- RSET <CRLF>
- SEND <SP> FROM:<reverse-path> <CRLF>
- SOML <SP> FROM:<reverse-path> <CRLF>
- SAML <SP> FROM:<reverse-path> <CRLF>
- VRFY <SP> <string> <CRLF>
- EXPN <SP> <string> <CRLF>
- HELP [<SP> <string>] <CRLF>
- NOOP <CRLF> QUIT <CRLF>
- TURN <CRLF>



## 扩展的SMTP

➢ SMTP的不足
- 不包括认证
- 传输ASCII而不是二进制数据
- 邮件以明文形式出现

FROM命令可以给出任何发件人地址

需要Base64 MIME编码方案，效率低下

无加密功能用于防止隐私窥探

| Keyword | Description |
|---|---|
| AUTH | Client authentication |
| BINARYMIME | Server accepts binary messages |
| CHUNKING | Server accepts large messages in chunks |
| SIZE | Check message size before trying to send |
| STARTTLS | Switch to secure transport (TLS; see Chap. 8) |
| UTF8SMTP | Internationalized addresses |

某些SMTP扩展功能

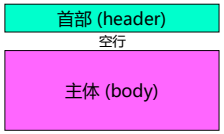


## 4. 邮件格式

➢ 首部（header）必须含有一个From:首部行和一个To:首部行，还可以包含 Subject: 等其他可选的首部行

➢ 注意这些首部行不同于前面前面说到的 SMTP 命令（即使那里也包含了某些相同的词汇，如from和to），SMTP中的命令是SMTP握手协议的一部分，而这里的首部行则是邮件报文自身的一部分

➢ 消息体（body）指邮件正文

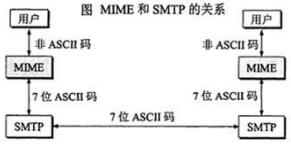



邮件基本格式



## 邮件格式

➢ RFC 5322—Internet邮件格式
- 基本ASCII电子邮件使用RFC 5322
- 是最新的Internet邮件格式，兼容RFC 822




➢ MIME(Multipurpose Internet Mail Extensions)—多用途Internet邮件扩展
- 由于SMTP只能传送一定长度的ASCII码邮件，其他非英语国家的文字(中文，俄文，日文)就无法传送，而且SMTP无法传送可执行文件以及其他二进制对象。因此提出了多用途网络邮件扩充。
- MIME并未改动SMTP，而是继续使用目前的格式，但是增加了邮件主题的结构，并定义了传送非ASCII码的编码规则。亦即，MIME邮件可在现有的电子邮件程序和协议下传送，但MIME邮件可传送的内容比SMTP要多。

## RFC 5322

- RFC 5322与邮件传输相关的头字段

| Header | Meaning |
|---|---|
| To: | Email address(es) of primary recipient(s) |
| Cc: | Email address(es) of secondary recipient(s) |
| Bcc: | Email address(es) for blind carbon copies |
| From: | Person or people who created the message |
| Sender: | Email address of the actual sender |
| Received: | Line added by each transfer agent along the route |
| Return-Path: | Can be used to identify a path back to the sender |

RFC 5322与邮件传输相关的头字段



## RFC 5322

- RFC 5322邮件头使用的某些字段

| Header | Meaning |
|---|---|
| Date: | The date and time the message was sent |
| Reply-To: | Email address to which replies should be sent |
| Message-Id: | Unique number for referencing this message later |
| In-Reply-To: | Message-Id of the message to which this is a reply |
| References: | Other relevant Message-Ids |
| Keywords: | User-chosen keywords |
| Subject: | Short summary of the message for the one-line display |

RFC 5322邮件头使用的某些字段



## MIME

- MIME沿用了RFC 822格式，但是加入了消息体结构，定义了非ASCII消息编码规则
- MIME类型的消息可以使用现有的邮件程序和协议进行传送，但是需要用户代理能够解释MIME格式

| Header | Meaning |
|---|---|
| MIME-Version: | Identifies the MIME version |
| Content-Description: | Human-readable string telling what is in the message |
| Content-Id: | Unique identifier |
| Content-Transfer-Encoding: | How the body is wrapped for transmission |
| Content-Type: | Type and format of the content |

MIME增加的消息头部



## MIME

- MIME编码方案

| 编码方法 | 说明 |
|---|---|
| 7bit | 7 位 ASCII 编码，每行不能超过 1000 个字符（包括回车和换行）。缺省编码方法。 |
| 8bit | 8 位非 ASCII 编码，每行不能超过 1000 个字节（包括回车和换行）。 |
| Binary | 8 位非 ASCII 编码，任意长度的字节串。 |
| Base64 | 将任意长度的字节串转换为用 7 位 ASCII 编码表示的字符串。可用于二进制和非文本数据的编码。 |
| Quoted-printable | 将任意长度的字节串转换为 ASCII 编码表示的字符串。可用于二进制和非文本数据的编码。 |

# MIME

➢ MIME内容类型和示例子类型

| Type | Example subtypes | Description |
|---|---|---|
| text | plain, html, xml, css | Text in various formats |
| image | gif, jpeg, tiff | Pictures |
| audio | basic, mpeg, mp4 | Sounds |
| video | mpeg, mp4, quicktime | Movies |
| font | otf, ttf Fonts for typesetting | |
| model | vrml | 3D model |
| application | octet-stream, pdf, javascript, zip | Data produced by applications |
| message | http, RFC 822 | Encapsulated message |
| multipart | mixed, alternative, parallel, digest | Combination of multiple types |



# MIME

```
From: alice@cs.washington.edu
To: bob@ee.uwa.edu.au
MIME-Version: 1.0
Message-Id: <0704760941.AA00747@cs.washington.edu>
Content-Type: multipart/alternative; boundary=qwertyuiopasdfghjklzxcvbnm
Subject: Earth orbits sun integral number of times

This is the preamble. The user agent ignores it. Have a nice day.

--qwertyuiopasdfghjklzxcvbnm
Content-Type: text/html
<p>Happy birthday to you<br>
Happy birthday to you<br>
Happy birthday dear <b> Bob </b><br>
Happy birthday to you</p>

--qwertyuiopasdfghjklzxcvbnm
Content-Type: message/external-body;
    access-type="anon-ftp";
    site="bicycle.cs.washington.edu";
    directory="pub";
    name="birthday.snd"

content-type: audio/basic
content-transfer-encoding: base64
--qwertyuiopasdfghjklzxcvbnm--
```

例：由HTML和音频多个部分组成的邮件




**总结：**

MIME是目前互联网电子邮件普遍采用的格式标准，包括了信息格式、媒体类型、编码方式等各方面的内容。按照OSI分层模型，MIME属于其中的【表示层】。

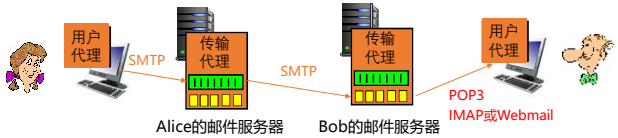


# 5. 最后传递

➢ 邮件已经到达Bob的邮箱，接下来的工作就是将邮件的一个副本传送到Bob的用户代理以便显示

➢ 最终交付（邮件访问）协议: 从邮件服务器的邮箱中获取邮件

- POP3：Post Office Protocol-Version 3，第三版邮局协议
- IMAP：Internet Message Access Protocol，Internet邮件访问协议
- Webmail（HTTP）：基于Web的电子邮件

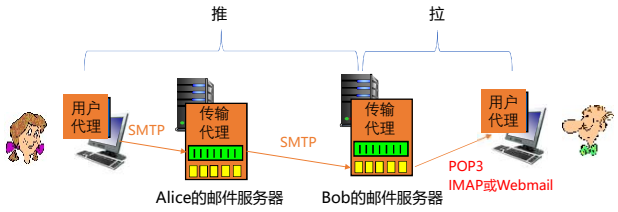





# 最后传递

➢ 为什么不能使用SMTP获取邮件?

- 接收方的用户代理不能使用 SMTP 从传输代理获取邮件，因为取邮件是一个拉操作，而 SMTP 是一个推协议，通过引入最终交付（邮件访问）协议来解决这个问题。



## （1）POP3协议

- POP3由RFC1939定义，是一个非常简单的最终交付协议
- 当用户代理打开一个到端口110上的TCP连接后，客户/服务器开始工作
- POP3的三个阶段：
  - 认证(Authorization)：处理用户登录的过程
  - 事务处理(Trnsactions)：用户收取电子邮件，并将邮件标记为删除
  - 更新(Update)：将标为删除的电子邮件删除

POP3使用客户/服务器工作方式，在接收邮件的用户PC机中必须运行POP客户程序，而在用户所连接的ISP的邮件服务器中则运行POP服务器程序



## POP3的三个阶段

1）认证阶段

- 客户命令:
  - user: 用户名
  - pass: 密码
- 服务器响应
  - +OK
  - -ERR

S: +OK POP3 server ready
C: user bob
S: +OK
C: pass hungry
S: +OK user successfully logged on



## POP3的三个阶段

2）事务处理阶段

- list: 邮件列表
- retr: 通过邮件号获取邮件
- dele: 删除

3）退出阶段

- quit

C: list
S: 1 498
S: 2 912
S: .
C: retr 1
S: <message 1 contents>
S: .
C: dele 1
C: retr 2
S: <message 1 contents>
S: .
C: dele 2
C: quit
S: +OK POP3 server signing off



## POP3 命令与响应

- POP3最小命令集
  - USER name
  - PASS string
  - QUIT
  - STAT
  - LIST [msg]
  - RETR msg
  - DELE msg
  - NOOP
  - RSET
  - QUIT
- 可选的POP3命令
  - APOP name digest
  - TOP msg n
  - UIDL [msg]
- POP3命令响应
  - +OK
  - -ERR

## （2）IMAP

- IMAP—Internet邮件访问协议[RFC 2060]
  - 用于最终交付的主要协议
  - IMAP是较早使用的最终交付协议—POP3的改进版
  - 邮件服务器运行侦听端口143的IMAP服务器
  - 用户代理运行一个IMAP客户端
  - 客户端连接到服务器并开始发出命令



## IMAP特性

- IMAP允许用户在不同的地方、使用不同的计算机、随时上网阅读和处理自己的邮件
- IMAP服务器把每个邮件与一个文件夹联系起来，当邮件第一次到达服务器时，它与收件人的INBOX文件夹相关联。收件人能够把邮件移到一个新的、用户创建的文件夹中，阅读邮件，删除邮件等
- IMAP还为用户提供了在远程文件夹中查询邮件的命令，按指定条件去查询匹配的邮件
- 与POP3不同，IMAP服务器维护了IMAP会话的用户状态信息，例如，文件夹的名字以及哪些邮件与哪些文件夹相关联



## IMAP特性

- IMAP具有允许用户代理获取邮件某些部分的命令
  - 例如，一个用户代理可以只读取一个邮件的首部，或只是一个多部分MIME邮件的一部分
- 用户代理和其邮件服务器之间使用低带宽连接（如一个低速调制解调器链路）的时候，用户可能并不想取回邮箱中的所有邮件，其要避免可能包含如音频或视频片断的大邮件
- IMAP的缺点是如果用户没有将邮件复制到自己的PC上，则邮件一直是存放在IMAP服务器上。因此用户需要经常与IMAP服务器建立连接



## IMAPv4命令

| Command | Description |
|---|---|
| CAPABILITY | List server capabilities |
| STARTTLS | Start secure transport (TLS; see Chap. 8) |
| LOGIN | Log on to server |
| AUTHENTICATE | Log on with other method |
| SELECT | Select a folder |
| EXAMINE | Select a read-only folder |
| CREATE | Create a folder |
| DELETE | Delete a folder |
| RENAME | Rename a folder |
| SUBSCRIBE | Add folder to active set |
| UNSUBSCRIBE | Remove folder from active set |
| LIST | List the available folders |
| LSUB | List the active folders |
| STATUS | Get the status of a folder |
| APPEND | Add a message to a folder |
| CHECK | Get a checkpoint of a folder |
| FETCH | Get messages from a folder |
| SEARCH | Find messages in a folder |
| STORE | Alter message flags |
| COPY | Make a copy of a message in a folder |
| EXPUNGE | Remove messages flagged for deletion |
| UID | Issue commands using unique identifiers |
| NOOP | Do nothing |
| CLOSE | Remove flagged messages and close folder |
| LOGOUT | Log out and close connection |

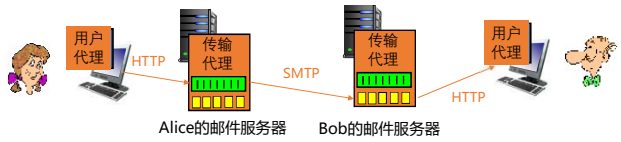
# POP3 vs IMAPv4

| Feature | POP3 | IMAP |
|---|---|---|
| Where is protocol defined | RFC 1939 | RFC 2060 |
| TCP port used | 110 | 143 |
| Where is e-mail stored | User's PC | Server |
| Where is e-mail read | Off-line | On-line |
| Connect time required | Little | Much |
| Use of server resources | Minimal | Extensive |
| Multiple mailboxes | No | Yes |
| Who backs up mailboxes | User | ISP |
| Good for mobile users | No | Yes |
| User control over downloading | Little | Great |
| Partial message downloads | No | Yes |
| Are disk quotas a problem | No | Could be in time |
| Simple to implement | Yes | No |
| Widespread support | Yes | Growing |



# Webmail

- Webmail——基于Web的电子邮件
  - 提供电子邮件服务的SMTP和IMAP替代方案
  - 使用Web作为界面，用户代理就是普通的浏览器
  - 用户及其远程邮箱之间的通信通过HTTP进行






# 目 录

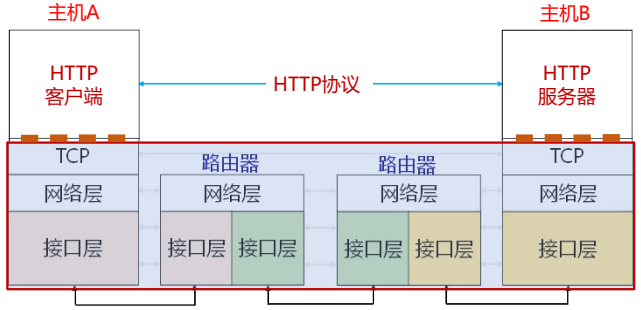

- 7.1 应用层概述（p3）
- 7.2 域名系统（p21）
- 7.3 电子邮件（p70）
- 7.4 WWW（p102）
- 7.5 其它应用层协议（p147）
- 7.6 流式音频和视频
- 7.7 内容分发

1. WWW体系结构概述
2. 静态Web（对象）
3. 动态Web和Web应用
4. HTTP协议
5. Web缓存技术与Web代理
6. Web安全与隐私



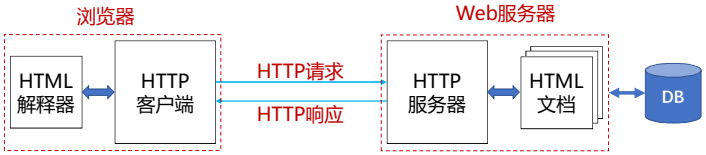

# 1. WWW体系结构与协议

- WWW = World Wide Web = 万维网
- HTTP 客户端和服务器之间执行的HTTP协议



# WWW体系结构与协议

- 服务器
  - Web页面（HTML文档）：包含到多种对象或链接
  - Web对象（包括：静态对象和动态对象）：可以是 HTML文档、 图像文件、视频文件、声音文件、脚本文件等
  - 对象用URL（统一资源定位符）编址：协议类型://主机名:端口/路径和文件名
- 客户端
  - 发出请求、接收响应、解释HTML文档并显示
  - 有些对象需要浏览器安装插件






# 统一资源定位器URLs

- 统一资源定位器URLs (Uniform Resource Locators)
  - 例如：http://www.phdcomics.com:8000/comics.php

协议类型　主机名即服务器　端口　路径和文件名

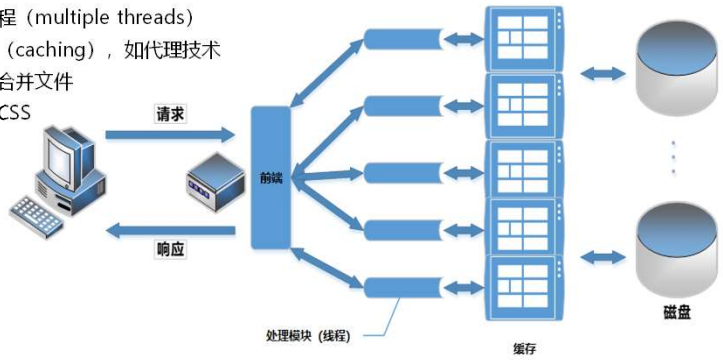
| 名字 | 用途 | 实例 |
|---|---|---|
| http | 超文本HTML | http://www.xaut.edu.cn/xxgk/xxjj.htm |
| https | 安全超文本 | https://www.overleaf.com/ |
| ftp | FTP | ftp://ftp.xaut.edu.cn |
| file | 本地文件 | file:///usr/xaut/prog.c |
| mailto | 发送邮件 | mailto:xinyzhang@swjtu.edu.cn |
| rtsp | 流媒体 | rtsp://youtube.com/montypython.mpg |
| sip | 多媒体呼叫 | sip:eve@advesary.com |



# WWW性能提升方法

前端（front end）

- 多线程（multiple threads）
- 缓存（caching），如代理技术
- 压缩合并文件
- 优化CSS






# 2. Web对象

- WEB网页对象
  - 静态对象与静态网页
    - 文本，表格，图片，图像和视频等多媒体类型的信息（实现语言：标记语言，如：HTML，XML，PHP等）
    - 页面字体、颜色和布局等风格类型（样式）的信息（实现语言：层叠样式表CSS）
  - 动态对象与动态网页
    - 交互信息，比如，用户注册信息、登录信息等，公告查询（实现：PHP/JSP等语言+MySQL等数据库）
  - 链接
    - 超链接（HyperLinks）

# 静态Web与HTML版本

| 特点 | HTML1.0 | HTML2.0 | HTML3.0 | HTML4.0 | HTML5.0 |
|---|---|---|---|---|---|
| 超链接、图像、列表 | √ | √ | √ | √ | √ |
| 活动地图和图像 | | √ | √ | √ | √ |
| 表单 | | √ | √ | √ | √ |
| 方程式 | | | √ | √ | √ |
| 工具条 | | | √ | √ | √ |
| 表格 | | | √ | √ | √ |
| 访问功能 | | | | √ | √ |
| 对象嵌入 | | | | √ | √ |
| 风格 | | | | √ | √ |
| 脚本 | | | | √ | √ |
| 视频和音频 | | | | √ | √ |
| 内联矢量图 | | | | | √ |
| XML表示 | | | | | √ |
| 后台线程 | | | | | √ |
| 浏览器存储 | | | | | √ |
| 画曲线 | | | | | √ |

```
<html>
  <head>
    <title>计算机网络 </title>
  </head>
  <body>
      ..........
  </body>
</html>
```





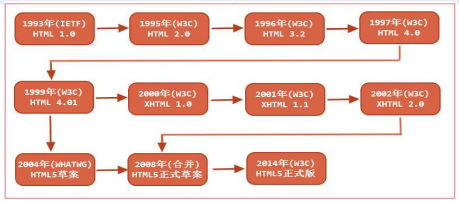

# HTML文档与静态网页






# HTML和XML

| 比较内容 | HTML | XML |
|---|---|---|
| 扩展性 | 无可扩展性，标记固定 | 元标记语言，可定义新的标记语言，标记可由用户定义 |
| 侧重点 | 侧重信息的表现形式 | 侧重结构化的信息描述 |
| 语法 | 不严格（嵌套、配对） | 严格要求嵌套、配对严格按照DTD(文件类型定义)的要求 |
| 可读性和可维护性 | 难于阅读，难于维护 | 结构清晰，便于阅读和维护 |
| 数据本身与显示 | 数据与显示通常合在一起 | 数据与显示分离 |
| 重用性 | 低 | 高 |



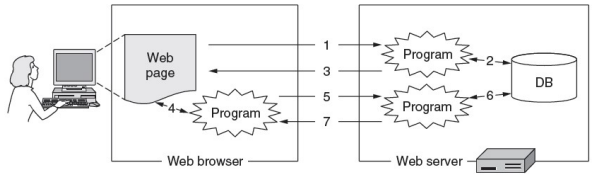



# 3. 动态Web概述

- 为什么需要动态Web?

  动态页面的内容一般都是依靠服务器端的程序来生成的，不同人、不同时候访问页面，显示的内容都可能不同；可实现更多功能；数据库为基础，降低网站维护工作量。

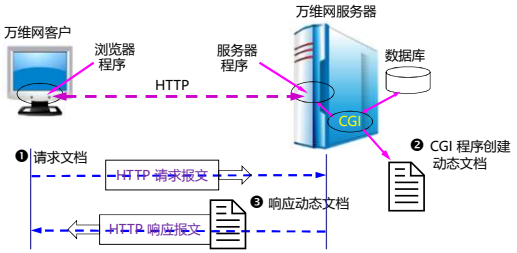
- 动态Web页面
  - 通用网关接口CGI (*.cgi)
  - 脚本语言+数据库技术 (*.php, *.asp, *.aspx)



# 通用网关接口CGI

- CGI (Common Gateway Interface) 是一种标准
  - 定义了动态文档应如何创建
  - 输入数据应如何提供给应用程序
  - 输出结果应如何使用

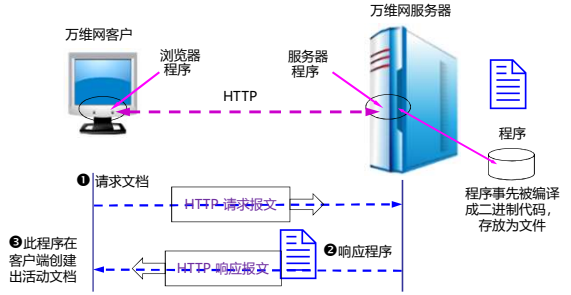
✓ “通用”：CGI 标准所定义的规则对其他任何语言都是通用的。
✓ “网关”：CGI 程序的作用像网关。
✓ “接口”：有一些已定义好的变量和调用等可供其他 CGI 程序使用。
✓ CGI 程序正式名字是 CGI 脚本 (script)。可采用php，jsp，asp，aspx等实现。






# 脚本语言+数据库技术

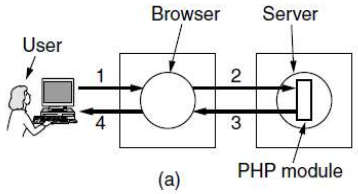
- “脚本”指的是一个程序，它被另一个程序（即解释程序，如Web浏览器）解释执行，而不是由计算机的处理器来解释或执行



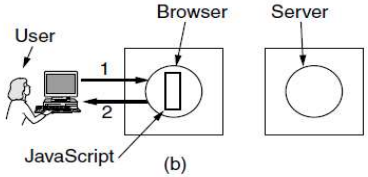



# 动态Web的执行

- 脚本程序的执行差异（在服务器端和客户端执行脚本的不同之处）
  - 好的动态Web开发形式：显示与脚本程序相分离（显示+脚本+CSS3）




PHP在服务器端




JavaScript脚本在客户端



# 典型的动态Web技术：AJAX

- AJAX：Asynchronous JavaScript and XML
  - 超文本标记语言HTML+CSS: 用于Web网页内容的显示
  - 文档对象模型DOM（Document Object Model）：文档对象模型（DOM）是HTML和XML文档的编程接口，其本质上是页面的API，允许程序读取和操作页面的内容、结构和样式；采用树形结构组织
  - 扩展标记语言XML（eXtensible Markup Language）：用于程序与服务器交换应用数据
  - 异步式工作方式：用于发送和检索XML数据
  - JavaScript：用于将以上所有功能进行组合，并协同工作
- AJAX是一种集成现有标准的新方法，用于创建更好更快以及交互性更强的Web应用程序的技术。是一种独立于 Web 服务器软件的浏览器技术。使用Ajax技术网页应用能够快速地将增量更新呈现在用户界面上，而不需要重载（刷新）整个页面，这使得程序能够更快地回应用户的操作。从而提高系统性能，优化用户界面。

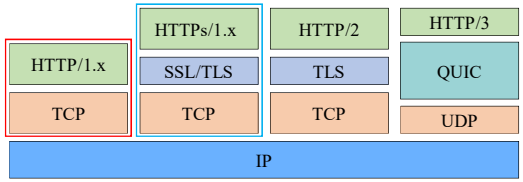
# 4. HTTP概述

- 超文本传输协议HTTP(HyperText Transfer Protocol)：传输层用TCP协议，缺省使用TCP的80端口
- HTTP为无状态协议：无状态是指协议对于事务处理没有记忆能力，意味着如果后续处理需要前面的信息，则它必须重传，这样可能导致每次连接传送的数据量增大。但在服务器不需要先前信息时它的应答就较快。为解决HTTP协议无状态，两种用于保持HTTP连接状态的技术：Cookie 和 Session。
  - 无状态协议：效率低、但简单
  - 有状态协议：维护状态相对复杂，需要维护历史信息，在客户端或服务器出现故障时，需要保持状态的一致性等
- HTTP标准
  - HTTP/1.0: RFC 1945（1996年）
  - HTTP/1.1: RFC 2616（1999年）
  - HTTP/2: RFC 7540（2015年）、RFC 8740（2020年）



# HTTP发展现状

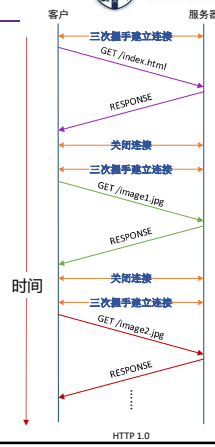
- HTTP/1.0（1996）
  - 无状态，非持久连接
- 与HTTP/1.1（1999）
  - 支持长连接和流水线机制
  - 缓存策略优化、部分资源请求及断点续传
- HTTPS：HTTP+TLS（2008）
  - 增加SSL/TLS（TLS 1.2）层，在TCP之上提供安全机制
- HTTP/2.0（2015、2020）
  - 目标：提高带宽利用率、降低延迟
  - 增加二进制格式、TCP多路复用、头压缩、服务端推送等功能






# HTTP1.0执行过程

- 假设用户输入URL
  - http://www.nankai.edu.cn/computer/index.html
  - 如该页面包含2幅jpg图像
    - 需要**执行三次完整的连接过程**（即：连接的建立，数据传输，连接终止），**包含三次TCP过程（含三次HTTP过程）**
      - 三次握手连接建立
      - Request （Get /index.html）
      - Respone
      - 关闭连接
      - 第一幅图像.....第二副图像 .....
  - 如若含100副图像呢?





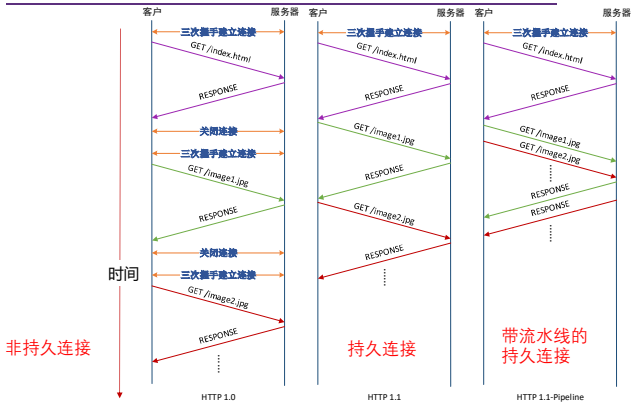

# 非持久连接和持久连接(HTTP1.x)

思考：持久连接中TCP连接何时关闭?

- 非持久连接
- HTTP/1.0缺省为非持久连接
  - ✓ 服务器接收请求、响应、关闭TCP连接
- 获取每个对象需要两阶段
  - ✓ 建立TCP连接
  - ✓ 对象请求和传输
- 每次连接需要经历TCP慢启动阶段
- 串行非持久和并行非持久连接

- 持久连接
- HTTP/1.1缺省为持久连接
  - ✓ 在相同的TCP连接上，服务器接收请求、响应；再接收请求、响应；响应后保持连接
- HTTP/1.1支持流水线机制
  - ✓ 需要按序响应
  - ✓ 经历较少的慢启动过程，减少往返时间
  - ✓ 降低响应时间

# HTTP 1.x比较






**实例：**

现有一个客户向服务器请求一个Web页面，假设这个Web页面里面含有一个HTML基本文件和10个JPEG图形，并且这11个对象位于同一台服务器上。现在要把网页和引用对象都取回来。

1）**采用非持续连接的串行方式**：总共需要22个RTT。

2）**采用非持续连接的并行方式**：假设最多可以并行3条TCP连接，总共需要10个RTT。首先建立TCP连接并传输HTML文件=2个RTT，10/3=3……1；可分成四组，前三组有3个(6个RTT)，最后一组只有1个JPEG文件(2个RTT)

3）**采用持续连接不带流水的方式**：总共需要12个RTT。首先建立TCP连接并传输HTML文件=2个RTT，接下来取10个JPEG文件

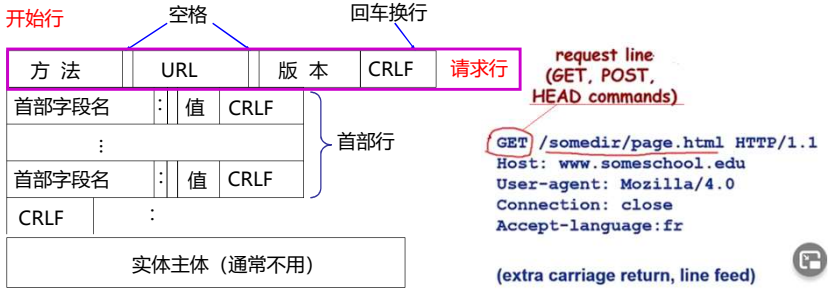
4）**采用持续连接带流水的方式**：总共需要3个RTT。带流水的持续连接就是一下子把所有的能发的请求都发出去，再一下子把要的东西都拿回来。建立TCP连接并传输HTML文件=2个RTT接下来取10个JPEG文件，10个请求同时发出去，10个JPEG文件同时取回来，共需要1个RTT。总共需要2+1=3个RTT。



# HTTP报文结构：请求报文

- 报文由三个部分组成，即开始行、首部行和实体主体
- 在请求报文中，开始行就是请求行

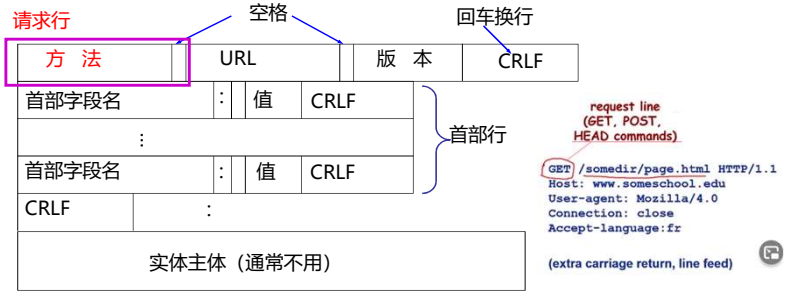
https://blog.csdn.net/weixin_40757930/article/details/114276757






# HTTP报文结构：方法（类型）

- 方法是对所请求的对象进行的操作，实际上也就是一些命令
- 请求报文的类型是由它所采用的方法决定的



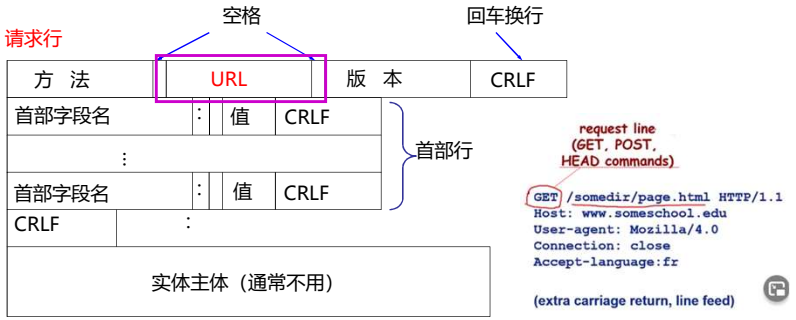
# HTTP报文结构：具体方法

| 方法或操作 | 含　义 |
|---|---|
| OPTION | 请求一些选项的信息 |
| GET | 请求读取由 URL所标志的信息 |
| HEAD | 请求读取由 URL所标志的信息的首部 |
| POST | 给添加信息（例如，注释） |
| PUT | 在指明的 URL下存储一个文档 |
| DELETE | 删除指明的 URL所标志的资源 |
| TRACE | 用来进行环回测试的请求报文 |
| CONNECT | 用于代理服务器 |



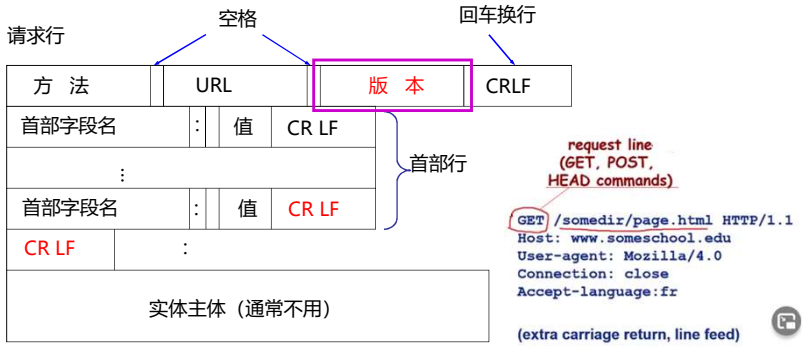
# HTTP报文结构：URL

➢ “URL”是所请求资源的URL



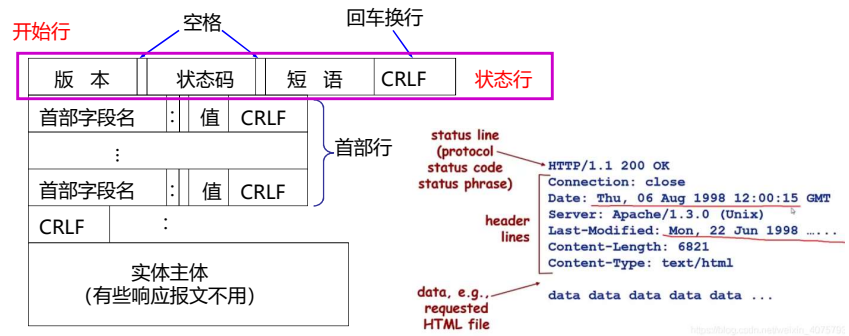



# HTTP报文结构：版本






# HTTP报文结构：响应报文



# HTTP响应报文：状态码

➢ 状态码都是三位数字

- 1xx 表示通知信息的，如请求收到了或正在进行处理。
- 2xx 表示成功，如接受或知道了。
- 3xx 表示重定向，表示要完成请求还必须采取进一步的行动。
- 4xx 表示客户的差错，如请求中有错误的语法或不能完成。
- 5xx 表示服务器的差错，如服务器失效无法完成请求。

➢ 典型的状态码

200 OK 请求成功，被请求的对象包含在该响应的数据部分

301 Moved Permanently 请求的对象被移走，新的位置在响应中通过Location: 给出

400 Bad Request 服务器不能解释请求报文

401 Unauthorized 错误代表用户没有访问权限，需要进行身份认证。

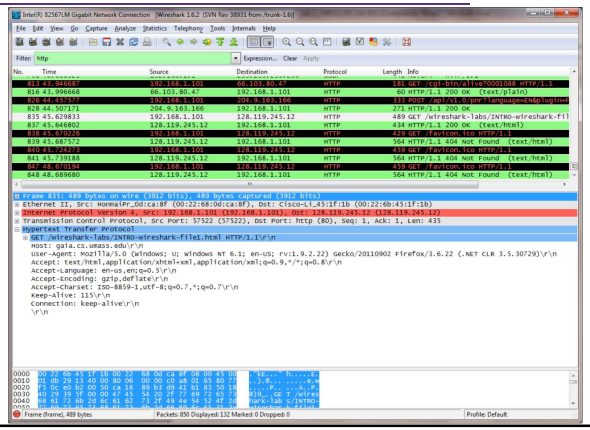
404 Not Found 服务器中找不到请求的文档

505 HTTP Version Not Supported

服务器不支持相应的HTTP版本



# Wireshark捕获的Web访问过程





# 5. Web缓存技术与Web代理

➢ 浏览器缓存

- 浏览器主机保存用户访问过的服务器Web页副本
- 再次访问该页，不必从服务器再次传输，提高访问效率

➢ Web代理服务器缓存

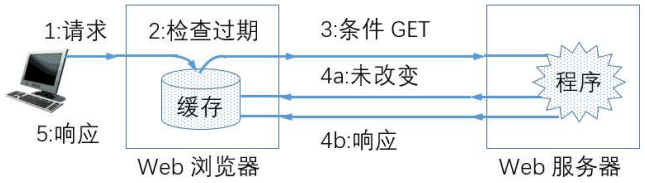
- ISP安装Web代理缓存服务器，保存客户访问过的服务器Web页副本
- 副本可以供ISP的所有客户访问，提高访问服务器的Web页效率



# Web缓存技术与Web代理

➢ 浏览器缓存

- 目标：再次访问缓存在浏览器主机中的Web页副本，不必从原始服务器读取
- 怎么保证Web页副本与原始服务器是一致的?



## Web缓存技术与Web代理

➢ Web代理服务器缓存

- 目标：代理服务器缓存已访问过的Web页副本，满足用户浏览器从代理服务器提取Web页，尽量减少原始服务器参与
- 设置用户浏览器，通过代理服务器进行Web访问
- 浏览器将所有的HTTP请求发送到代理服务器
  - ✓如果缓存中有被请求的对象，则直接返回对象
  - ✓否则，代理服务器向原始服务器请求对象，再将对象返回给客户端



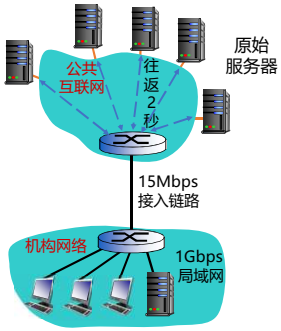

降低时延、减少网络流量



## Web缓存技术性能分析

➢ Web代理服务器缓存性能分析

- 假设:
  - 平均每个Web页对象大小: 1M bits
  - 机构网络内用户向原始服务器请求对象速率为平均每秒15个请求:15/sec
  - 互联网访问原始Web页的平均往返时延为2秒
  - 接入链路速率为15Mbps
- 结果:
  - 机构网络内带宽使用率1.5%，接入链路的使用率接近1
  - 总的时延：= 互联网时延+接入链路时延+ 机构网络时延 = 2 sec + minutes + msecs(机构网络时延以毫秒记)

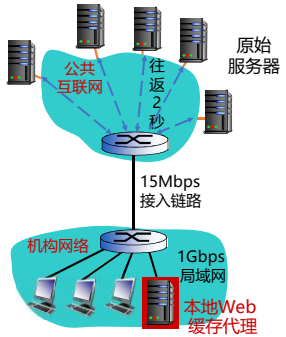





## Web缓存技术性能分析

➢ Web代理服务器缓存性能分析

- 假设:
  - 缓存代理访问命中率为40%
  - 平均每个Web页对象大小: 1M bits
  - 机构网络内用户向原始服务器请求对象速率为平均每秒15个请求:15/sec
- 结果:
  - 机构网络内带宽使用率1.5%，接入链路使用率接近0.6
  - 总的时延：= 0.6 * (互联网延迟) +0.4 * (Web代理命中率的延迟)
    = 0.6 (2秒+计10毫秒接入链路延迟) + 0.4 (计10毫秒机构网络内延迟)
    = ~ 1.2 secs






## Web缓存同步技术

➢ Web缓存存在的困难，即Web缓存与原始服务器Web页一致性问题

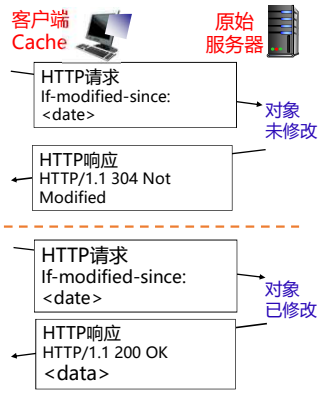
- 启发式策略：服务器响应页的Last-Modified头和Expires头启发。表示原始Web页没更改，最近一段时间也不会更改。Expires头很少用
- 询问式策略：通过特殊关键字头询问原始服务器，Web副本是否已更新

常用的Web代理服务器软件及特点:

| 软件名称 | 性能 | 功能 | 过滤规则配置 |
|---|---|---|---|
| Squid | 不能多核是硬伤，磁盘缓存容量有优势，性能中等 | 多，支持ACL角色控制，也支持ICP缓存协议 | 支持外部规则文件读取及热加载，支持热启动 |
| Varnish | 多核支持，内存缓存，性能强 | 够用，不支持集群，支持后端存活检查 | 不支持外部文件读取，需要转义，支持热启动 |
| Nginx | 多核支持，支持代理插件，性能较强 | 多，通过插件可以充当多角色服务器 | 不支持外部文件读取，需要转义，支持热启动 |
| ATS | 多核支持，磁盘/内存缓存，性能强 | 够用，支持插件开发，也支持ICP协议 | 支持外部规则文件读取及热加载，支持热启动，但缺乏文档 |
| HAProxy | 多核支持，无缓存，HTTP头支持语法操作，性能强 | 少，只专注HTTP头部解析和转发功能，支持ACL角色控制，支持后端存活检查 | 支持外部规则文件读取及热加载，支持热启动，支持会话粘滞和长连接 |

## Web代理与服务器同步

- 询问式策略：通过特殊的关键字头询问原始服务器，Web代理对应的原始Web页是否已更新
  - 客户端：在发送的HTTP请求中指定缓存的时间，请求头包含If-modified-since: <date>
  - 服务器：如果缓存的对象是最新的，在响应时无需包含该对象，响应头包含HTTP/1.1 304 Not Modified
    否则服务器响应HTTP/1.1 200 OK <data>
  - 或者服务器向客户端返回Web页时带一个ETag: <页内容的标签>
  - 客户端：请求中需要包含If-None-Match头让服务器比对If-None-Match :<Web页副本内容的标签>
- 原始服务器明确指令限制缓存某些Web页
  - 服务器返回Web页时，带一个no-cache禁止缓存
  - 需要授权访问的Web页也限制缓存





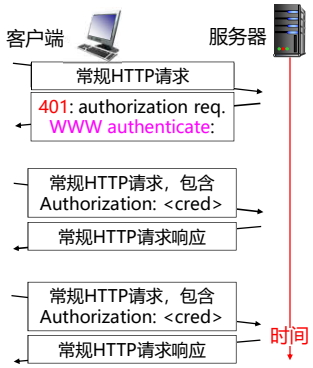

## 6. Web安全与隐私：访问安全

- Web访问安全
  - 并非所有Web页都会向公众开放
  - Web服务器可以限定客户端访问的IP地址空间，比如限制只向公司内部员工开放
  - Apache服务器将设置限制访问规则的文件.htaccess放置在被限制访问的页面所在的目录，客户端访问时进行规则匹配
  - 认证方法：在浏览器客户端的HTTP请求中给出“用户名-密码”，服务器进行HTTP验证



## Web安全与隐私：访问安全

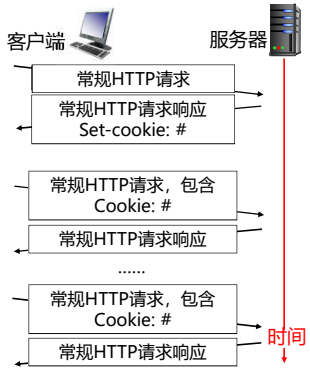
- Web访问安全
  - 无状态：http是不会为了下一次连接而维护本次连接所传输的信息。客户端需要在每个请求中携带认证信息
  - 认证方法：通常在HTTP请求中使用“用户名-密码”
  - 每个请求头中包含关键字authorization:
  - 如果请求头中无authorization: ，则服务器拒绝访问，并在响应头中包含
    WWW authenticate:



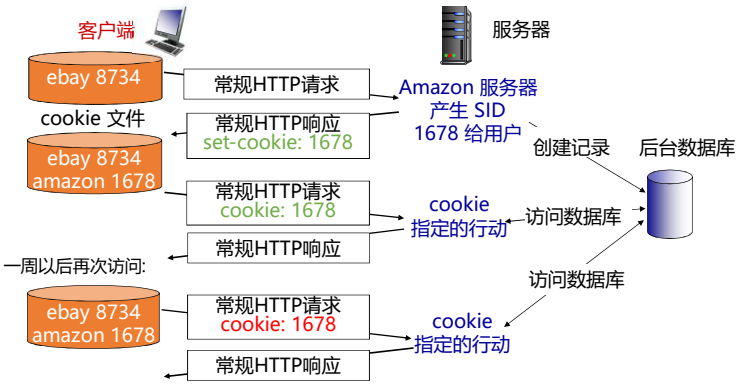



## Web安全与隐私：Cookie

- HTTP无状态协议，服务器用cookies保持用户状态
  - HTTP在响应的首部行里使用一个关键字头set-cookie：选择的cookie号具有唯一性
  - 后继的HTTP请求中使用服务器响应分配的cookie：
  - Cookie文件保存在用户的主机中，内容是服务器返回的一些附加信息，由用户主机中的浏览器管理
  - Web服务器建立后端数据库，记录用户信息，cookie作为关键字
  - 例如：
    - Set-Cookie: SID=31d4d96e407aad42; Path=/; Domain=example.com
    - Cookie: SID=31d4d96e407aad42



# Web安全与隐私：Cookie






# Web安全与隐私：Cookie

- Cookies通常包含5个字段
  - 域(domain)指明Cookie来自何方，可以访问该cookie的域名
  - 路径(path)标明服务器的文件树中哪些部分可以使用该Cookie：domain 和 path 一起限制了 cookie 能够被哪些 url 访问
  - 内容采用“名字=值”的形式，是Cookie存放内容的地方，可以达到4K容量，内容只是字符串，不是可执行程序
  - 过期expires 指定了 cookie 失效的时间
  - 安全指示浏览器只向使用安全传输连接的服务器返回Cookie

| 域 | 路径 | 内容 | 过期 | 安全 |
|---|---|---|---|---|
| Toms-casino.com | / | CustomerID=297793521 | 15-10-1017:00 | 是 |
| Jills-store.com | / | Cart=1-00501;1-07031;2-13721 | 11-1-1114:22 | 不 |
| Aportal.com | / | Prefs=Stk:CSCO+ORCL;Spt:Jets | 31-12-2023:59 | 不 |
| Sneaky.com | / | UserID=4627239101 | 31-12-1923:59 | 不 |



# Web安全与隐私：Cookie

- Cookie技术是把双刃剑，能分析用户喜好，向用户进行个性化推荐
  - 用Cookie在某网站标识用户信息，查找用户以前浏览网站记录
  - 用Cookie记录用户购物清单
  - 用Cookie可以保存4K内容，跟踪用户浏览网站的喜好
  - 用Cookie跨站点跟踪用户点击广告
- Cookie技术是把双刃剑，也能跟踪用户网络浏览痕迹，泄露用户隐私
  - Cookie跟踪用户以前浏览过哪些网站，跟踪用户频繁浏览哪类网站
  - Cookie收集用户信息，用户网络交互时关注的关键词
- “Cookie容易嵌入间谍程序？”，这是个误区，Cookie文件保存的只是文本串，没有可执行程序
  - 用户可以设置浏览器限制使用Cookie



# WWW的交互性和安全性

- WWW服务的交互性（可记录？可认证？个性化服务？安全性？）
  - Cookies
    - 如何控制注册或付费用户？如何跟踪购物车里的内容？如何提供个性化的服务？
  - 安全性：HTTPS与SSL

| 域 | 路径 | 内容 | 过期 | 安全 |
|---|---|---|---|---|
| Toms-casino.com | / | CustomerID=297793521 | 15-10-1017:00 | 是 |
| Jills-store.com | / | Cart=1-00501;1-07031;2-13721 | 11-1-1114:22 | 不 |
| Aportal.com | / | Prefs=Stk:CSCO+ORCL;Spt:Jets | 31-12-2023:59 | 不 |
| Sneaky.com | / | UserID=4627239101 | 31-12-1923:59 | 不 |

Cookies的可能表现

# 目 录


- 7.1 应用层概述（p7）
- 7.2 域名系统（p25）
- 7.3 电子邮件（p74）
- 7.4 WWW（p106）
- 7.5 其它应用层协议（p144）
- 7.6 流式音频和视频（）
- 7.7 内容分发网络（）

1. Telnet
2. SSH
3. FTP
4. TFTP
5. SNMP
6. 常见应用层协议




# 1. Telnet

- 远程登录是网络最早提供的基本服务之一，通过终端仿真协议实现对远程计算机系统的访问，就像访问本地资源一样，而且这个过程对用户是透明的
- 远程登录需要解决异构计算机系统的差异性问题，主要体现在对终端键盘输入命令的解释上
- Telnet协议最早出现在20世纪60年代后期，1983年由RFC 854确定为Internet标准
- Telnet协议使用C/S方式实现。在本地系统运行Telnet客户进程，在远程主机运行Telnet服务器进程
- Telnet协议使用TCP连接通信，服务器进程默认监听TCP23端口，服务器使用主进程等待新的请求，并产生从属进程来处理每一个连接

**Telnet**



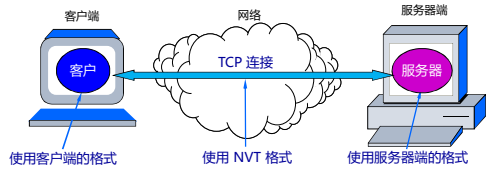
# Telnet

- Telnet协议引入网络虚拟终端NVT（Network Virtual Terminal），使用一种专门的键盘定义来屏蔽不同计算机系统对键盘输入的差异性，同时定义客户进程与远程服务器进程之间的交互过程
- NVT是Telnet协议定义的一组通用字符集，通过这种统一的数据表示方式，来保证不同硬件、软件与数据格式的终端与主机之间通信的兼容性
- 本地终端输入的字符首先由本地Telnet客户进程转换为NVT格式，通过网络将NVT格式的字符传输到远程主机，远程Telnet服务器进程再将NVT格式的字符转换为远程主机能够识别和处理的字符格式
- 使用Telnet协议在网络中传输的数据都是NVT格式，不同的用户终端与服务器进程均与本地终端格式无关



# Telnet

- Telnet的工作过程
  - 本地Telnet客户进程与远程主机上的Telnet服务器进程建立TCP连接
  - 将本地终端上输入的用户名和口令及以后输入的任何命令或字符以网络虚拟终端NVT格式传输给远程主机
  - 将远程主机输出的NVT格式的数据转化为本地所接受的格式送回本地终端，包括输入命令回显和命令执行结果
  - 本地终端对远程主机撤销连接，从而结束 Telnet远程登录过程



## 2. SSH

- SSH（Secure Shell）和Telnet都是远程连接控制协议，在网络技术的应用中常被用于远程连接控制服务器、路由器和交换机，方便工程师远程对设备进行运营维护。SSH目前有两个版本：SSH1和SSH2，两者互相不兼容，连接时主要取决于主机安装的连接软件版本。
- SSH和Telnet的主要区别有以下两点：

1）SSH连接时采用密文传输数据，而Telnet则采用明文传输数据。同时在使用SSH协议时，需要匹配服务端生成的秘钥才能连接成功；而telnet则不需要匹配秘钥。

2）SSH连接端口为22，Telnet连接端口为23。

3）SSH协议支持远程命令执行，文件传输和安全隧道；Telnet只能进行远程终端登录和命令执行。



## 3. FTP

- 网络环境下复制文件的复杂性
  - 计算机存储数据的格式不同
  - 文件目录结构和文件命名规则不同
  - 对于相同的文件存取功能，操作系统使用的命令不同
  - 访问控制方法不同
- 文件传输协议FTP(File Transfer Protocol)是Internet上使用最广泛的应用层协议之一
  - FTP提供交互式的访问，允许用户指明文件的类型与格式，并允许文件具有存取权限
  - FTP屏蔽了各计算机系统的细节，适用于在异构网络中任意计算机之间传送文件
  - RFC 959早在1985年就已经成为Internet的正式标准



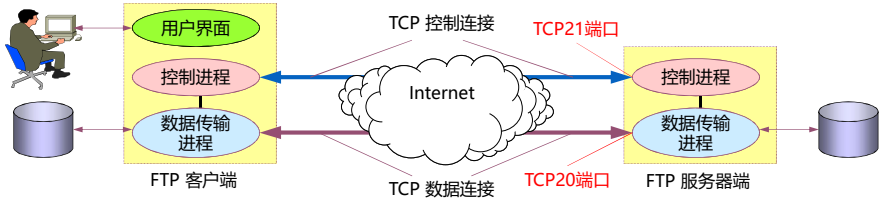
## FTP工作过程

- FTP使用C/S方式实现
- FTP工作过程
  - 服务器主进程打开TCP21端口，等待客户进程发出的连接请求
  - 客户可以用分配的任意一个本地端口号与服务器进程的TCP21端口进行连接
  - 客户请求到来时，服务器主进程启动从属进程来处理客户进程发来的请求
  - 服务器从属进程对客户进程的请求处理完毕后即终止。但从属进程在运行期间根据需要还可能创建其他一些子进程
  - 服务器主进程返回，继续等待接收其他客户进程发来的连接请求，服务器主进程与从属进程并行工作



## FTP功能

- FTP的两个端口与两个连接
  - 控制连接在整个会话期间一直保持，客户进程发出的文件传输请求通过控制连接发送给服务器控制进程（工作在TCP21端口），但控制连接不用来传输文件；
  - 服务器控制进程在接收到客户进程发送来的文件传输请求后就创建数据传输进程（工作在TCP20端口）和数据连接；
  - 数据连接用来连接客户进程和服务器数据传输进程，实际完成文件的传输。服务器数据传输进程在文件传输完毕后关闭数据连接并结束运行；



## Ftp命令集：

```
ftp> ?
Commands may be abbreviated.  Commands are:
!               delete          literal         prompt          send
?               debug           ls              put             status
append          dir             mdelete         pwd             trace
ascii           disconnect      mdir            quit            type
bell            get             mget            quote           user
binary          glob            mkdir           recv            verbose
bye             hash            mls             remotehelp
cd              help            mput            rename
close           lcd             open            rmdir
ftp> _
```

① pwd 查看用户当前所在的路径；

② ls 列出服务器上提供的软件目录；

③ cd 改变用户所在服务器的目录；

④ get 将ftp服务器上的文件下载到客户机的当前目录；

⑤ mget *.* 将ftp服务器上当前目录下的所有文件全部下载到客户机的当前目录；

⑥ put wrar260.exe 将本地客户机的wrar260.exe传到服务器上去；

⑦ mput *.* 将当前目录下的所有文件全部上传导服务器；

⑧ remotehelp 查看远方服务器准许用户执行哪些命令，屏幕会显示很多命令，这些是远程机器上边的命令，如果要执行，只要在命令前加上一个quote就可以，如：

ftp> *quote syst*

215 UNIX Type: L8

⑨ lcd 改变客户机的本地目录；

⑩ quit 退出ftp实用程序。



## FTP的主动模式和被动模式

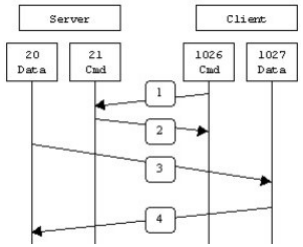
### 1. 主动模式(port)

1) 客户端与FTP服务器的命令端口21建立连接，并发送命令“PORT 1027”。

2) FTP服务器给客户端的命令端口返回一个“ACK”。

3) FTP服务器发起一个从数据端口(20)到客户端指定的数据端口(1027)的连接。

4) 客户端给服务器端返回一个"ACK"。




主动模式FTP的主要问题在于客户端。FTP客户端并没有实际建立一个到服务器数据端口的连接，它告诉服务器自己监听的端口号，服务器再回来连接客户端这个指定的端口。对于客户端的防火墙来说，这是从外部系统建立到内部客户端的连接，这通常会被阻塞的。



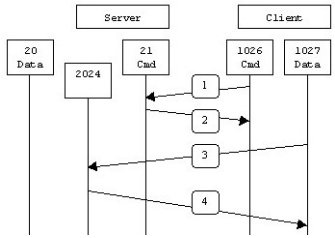
### 2. 被动模式（PASV）

1) 客户端与服务器命令端口21建立连接，并发送命令“PASV”。

2) 服务器返回命令“PORT 2024”，告诉客户端(服务器)用哪个数据连接侦听端口。

3) 客户端初始化一个从自己数据端口到服务器端指定数据端口2024的数据连接。

4) 给客户端的数据端口返回一个"ACK"响应。




* 被动模式FTP解决了客户端许多问题（如防火墙，NAT等），但给服务器端带来了更多问题。最大问题是需要允许客户端到服务器高位端口的连接。许多FTP守护程序(Wu-ftpd)允许管理员指定FTP服务器使用的端口范围。

* 客户端有的支持被动模式，有的不支持被动模式。



## 4. TFTP

- 简单文件传输协议 TFTP
  - TFTP(Trivial File Transfer Protocol)是一个很小且易于实现的文件传输协议
  - 使用C/S方式和UDP协议实现
  - 只支持文件传输而不支持交互
  - 没有庞大的命令集，没有列目录的功能，也不能对用户进行身份鉴别
  - 支持ASCII 码或二进制传送
  - 支持对文件进行读或写
  - 使用很简单的首部
- TFTP的数据传输
  - 数据PDU也称为文件块(block)，每个块按序从1开始编号
  - 每次传送的数据PDU中有512字节的数据，但最后一次可不足512字节
  - 若文件长度恰好为512字节的整数倍，则在文件传输完毕后，还必须在最后发送一个只含首部而无数据的数据 PDU
  - 若文件长度不是512字节的整数倍，则最后传送数据PDU的数据字段一定不满512字节，这正好可作为文件结束的标志

## TFTP

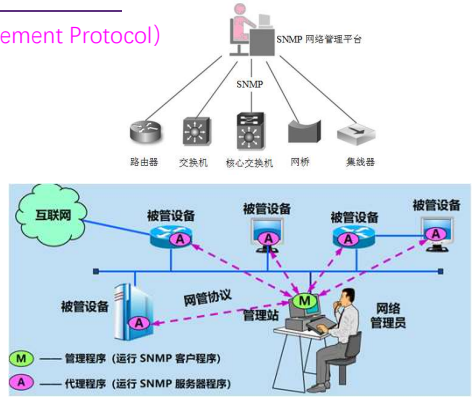
- TFTP 的工作过程
  - TFTP的工作过程类似停止-等待协议。每发送完一个文件块后就等待对方的确认，确认时应指明所确认的块编号
  - 发完数据后在规定时间内收不到确认就要重发该数据PDU
  - 发送确认PDU的一方若在规定时间内收不到下一个文件块，也要重发确认PDU。这样就能保证文件的传输不会因某一个数据PDU的丢失而失败
  - 开始工作时，TFTP客户进程发送一个读请求PDU或写请求PDU给TFTP服务器进程，其UDP端口号为69
  - TFTP服务器进程要选择一个新的端口和TFTP客户进程进行通信



## 5. SNMP

SNMP：Simple Network Management Protocol)

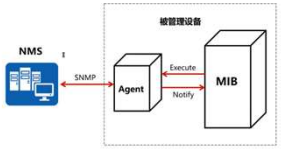
- 网络管理的目的
  - 有效利用网络资源
  - 及时报告和处理网络故障
  - 保障网络正常、高效运行
- 网络管理的功能
  - 配置管理（Configuration）
  - 故障管理（Fault）
  - 性能管理（Performance）
  - 计费管理（Accounting）
  - 安全管理（Security）






## SNMP

- 网络管理系统(NMS)
  - 管理器(Manager)：网络管理进程，提供网络管理用户界面，完成管理任务
  - 管理对象(Managed Object)：网络中的软硬件系统
  - 代理（Agent）
    - ✓ 管理代理(Management Agent)：管理对象中的管理进程
    - ✓ 外部代理(Proxy Agent)：管理对象外部增加的管理进程
  - 管理信息库(MIB，Management Information Base)
    - ✓ 本地MIB：每个代理管理自己的本地MIB，仅包含本地设备相关信息
    - ✓ 网络MIB：代理与管理器交换网络状态信息，共同构成整个网络的MIB
  - 网络管理协议（Network Management Protocol）
    - ✓ TCP/IP的简单网络管理协议SNMP（Simple Network Management Protocol）
    - ✓ OSI的公共管理信息协议CMIP（Common Management Information Protocol）






## SNMP

- SNMP概述
  - 指导思想
    - 尽可能简单：用简单的系统结构和协议来解决复杂的网络管理问题
      - ✓ 协议交互过程简单，仅有4类操作对网络进行管理
      - ✓ 使用UPD协议，以简化设计和降低通信开销
    - 影响尽量小：网络中任何软硬件的增减对原有管理对象的影响尽量小
  - 版本变迁
    - SNMPv1：RFC 1067(1988)、RFC 1098(1989)、RFC 1155~1157(1990)
    - SNMPsec：RFC 1351~1353(1992)
    - SNMPv2：RFC 1441~1445(1993)
    - SNMPv3：RFC 3410~3418(2002)，安全性和可管理性大为提高

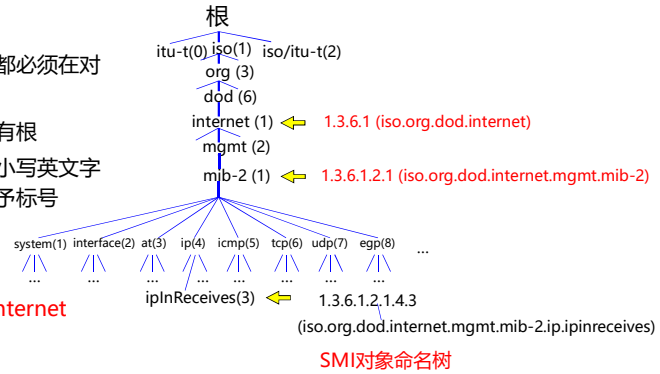
# SNMP

- SNMP实现
  - 管理信息结构SMI（Structure of Management Information）
    - 管理对象如何命名：对象命名树（Object Naming Tree）
    - 用来存储管理对象的数据类型有哪些：简单类型，结构化类型；收发双方使用基本编码规则BER(Basic Encoding Rule)说明每种数据类型中每个数据的值的表示
    - 在网络上传送的管理数据如何编码：使用抽象语法记法 1（ASN.1）来定义，ASN.1记法严格，使得数据的含义不存在二义性
  - 管理信息库MIB
    - 只有在MIB中的对象才是SNMP能够管理的
  - 协议规则
    - 轮询：读操作get，写操作set



# SNMP

- 对象命名树
  - 所有管理对象都必须在对象命名树上
  - 对象命名树没有根
  - 节点标识符用小写英文字母表示，并赋予标号
- 节点的表示
  - 标识符：iso.org.dod.internet
  - 标号：1.3.6.1




SMI对象命名树



# SNMP

| 分类 | 数据类型名称 | 含义 |
|---|---|---|
| 简单类型 | INTEGER | 任意长度整数 |
| | BIT STRING | 0 位或多位组成的二进制串 |
| | OCTET STRING | 0 位或多位组成的字节串 |
| | NULL | 空值 |
| | OBJECT IDENTIFIER | 定义的数据类型 |
| 结构化类型 | SEQUENCE | 由多个数据类型按序组成的值 |
| | SEQUENCE OF | 由同一数据类型按序组成的值 |
| | CHOICE | 可以从多个数据类型中选择一个 |
| | ANY | 任何数据类型 |

ASN.1 部分数据类型



# SNMP

| 组 名 | 组标识符 | 所包含的信息 |
|---|---|---|
| system | 1.3.6.1.2.1.1 | 与主机或路由器的操作系统相关的对象 |
| interfaces | 1.3.6.1.2.1.2 | 与网络接口相关的对象 |
| address translation | 1.3.6.1.2.1.3 | 与地址转换相关的对象 |
| ip | 1.3.6.1.2.1.4 | 与IP协议运行相关的对象 |
| icmp | 1.3.6.1.2.1.5 | 与ICMP协议运行相关的对象 |
| tcp | 1.3.6.1.2.1.6 | 与TCP协议运行相关的对象 |
| udp | 1.3.6.1.2.1.7 | 与UDP协议运行相关的对象 |
| egp | 1.3.6.1.2.1.8 | 与EGP协议运行相关的对象 |

MIB对象组

# SNMP

| MIB 变量 | 所属类别 | 意　义 |
|---|---|---|
| sysUpTime | system | 距上次重启动的时间 |
| ifNumber | interfaces | 网络接口数 |
| ifMtu | interfaces | 特定接口的最大传送单元 MTU |
| ipDefaultTTL | ip | IP 在生存时间字段中使用的值 |
| ipInReceives | ip | 接收到的数据报数目 |
| ipForwDatagrams | ip | 转发的数据报数目 |
| ipOutNoRoutes | ip | 路由选择失败的数目 |
| ipReasmOKs | ip | 重装的数据报数目 |
| ipFragOKs | ip | 分片的数据报数目 |
| ipRoutingTable | ip | IP 路由表 |
| icmpInEchos | icmp | 收到的 ICMP 回送请求数目 |
| tcpRtoMin | tcp | TCP 允许的最小重传时间 |
| tcpMaxConn | tcp | 允许的最大 TCP 连接数目 |
| tcpInSegs | tcp | 已收到的 TCP 报文段数目 |
| udpInDatagrams | udp | 已收到的 UDP 数据报数目 |

MIB变量举例



# SNMP

- SNMP协议规则——轮询方式
  - 管理器采用轮询方式，周期性“读” “写”来实现网络管理功能
    - ✓ “读”操作：管理器向代理发送get报文来检测管理对象状态
    - ✓ “写”操作：管理器向代理发送set报文来修改管理对象状态
  - 轮询方式的优点
    - ✓协议实现相对简单
    - ✓能限制所产生管理信息的通信量
  - 轮询方式的缺点
    - ✓不够灵活，而且所能管理的设备数目不能太多
    - ✓开销比较大



# SNMP

- SNMP协议规则——陷阱方式
  - SNMP允许代理不经过管理器询问就能发送某些信息，这种信息称为陷阱（trap）报文，通常代表代理检测到的重要事件
  - 当管理对象的代理检测到有事件发生时，就检查其门限值。代理只向管理器报告达到特定门限值的事件，这个过程称为过滤
  - 陷阱方式的优点
    - 仅在严重事件发生时才发送陷阱报文
    - 陷阱信息很简单且所需字节数很少

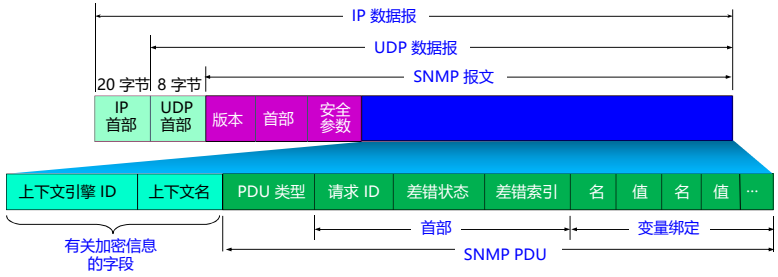


# SNMP

| 操作类型 | 说明 | SNMPv3报文 |
|---|---|---|
| 读 | 管理器读取管理对象状态信息的请求报文 | GetRequest-PDU<br>GetNextRequest-PDU<br>GetBulkRequest-PDU |
| 写 | 管理器修改管理对象状态信息的请求报文 | SetRequest-PDU |
| 响应 | 代理对管理器请求的应答报文 | Response-PDU |
| 通知 | 代理向管理器报告重要事件的报文 | Trapv2-PDU<br>InformRequest-PDU |

SNMPv3的报文类型

## SNMP




SNMP的报文格式



## SNMP

- SNMP协议实现
  - SNMP使用无连接的UDP协议实现，众所周知UDP协议不保证可靠交付
  - SNMP使用C/S方式实现
    - ✓代理运行服务器端进程在UPD端口161接收get或set报文以及发送response报文，与161端口通信的客户端进程则使用临时端口
    - ✓管理器运行客户端进程在UPD端口162接收来自代理的trap报文
- SNMP是有效的网络管理协议
  - 使用周期性轮询方式实现对网络资源的实时监控与管理
  - 使用陷阱方式报告重要事件的发生



## 6. 常见的应用层协议

| 应用 | 应用层协议 | 端口号 | 传输层协议 | 备注 |
|---|---|---|---|---|
| 域名解析 | DNS | 53 | UDP/TCP | 长度超过 512 字节时使用 TCP |
| 动态主机配置协议 | DHCP | 67/68 | UDP | |
| 简单网络管理协议 | SNMP | 161/162 | UDP | |
| 文件传送协议 | FTP | 20/21 | TCP | 控制连接 21，数据连接 20 |
| 远程终端协议 | TELNET | 23 | TCP | |
| 超文本传送协议 | HTTP | 80 | TCP | |
| 简单邮件传送协议 | SMTP | 25 | TCP | |
| 邮件读取协议 | POP3 | 110 | TCP | |
| 网际报文存取协议 | IMAP | 143 | TCP | |



## 本章内容

7.1 应用层概述（p7）
7.2 域名系统（p25）
7.3 电子邮件（p74）
7.4 WWW（p106）
7.5 其它应用层协议（p147）
7.6 流式音频和视频（168）
7.7 内容分发网络（186）

1. 流媒体概述
2. 数字音视频与编码
3. 流式存储媒体
4. 直播与实时音视频
5. 流媒体动态自适应传输

# 1. 流媒体概述

- 常见的流媒体服务
  - 点播：提前录制好，边下载边播放（起始时延<10s；类VCR操作（例如拖动进度条）<1~2s）
  - 直播：边录制边上传，边下载边播放（大规模直播往往有数秒的时延）
  - 实时交互：双方或多方实时交互式通信（时延<400ms可接受，VR则需要<25ms）

视频点播　　视频直播　　实时交互



# 流媒体概述

- 流媒体概念
  - 连续媒体（音视频）经压缩编码、数据打包后，经过网络发送给接收方
  - 接收方对数据进行重组、解码和播放
- 流媒体的特性
  - 端到端时延约束
  - 时序性约束：流媒体数据必须按照一定的顺序连续播放
  - 具有一定程度的容错性：丢失部分数据包也可完成基本功能
- 流媒体面临的挑战
  - 约束条件：网络特性（带宽有限、动态变化、延迟与抖动、丢失、异构性）
  - 目标：流媒体服务质量要素（画质、启动延迟、平滑、交互性）
  - 如何在“尽力服务”的网络传输条件下获得良好的视频质量？



# 流媒体概述




期望：高清、低延迟、不卡顿？可是网络不稳定☹

场景：点播、直播、会议、连麦、云游戏、VR、……

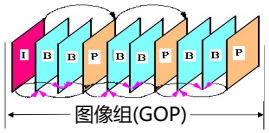


# 2. 数字音视频与编码

- MPEG视频压缩
  - 摄像机固定不变，演员慢慢走来走去：从前一帧中减去当前帧，对两帧之差运行JPEG算法
  - 摄像机在推拉移动：需要某种方法来补偿这种运动，这正是MPEG擅长的
- MPEG的输出包括3类帧
  - 帧内编码帧（I帧）：包含了压缩的静止图片（帧内编码，用JPEG来压缩静止图像）
  - 预测帧（P帧）：是与前一帧的逐块差值（帧间编码，消除跨帧的冗余度）
  - 双向帧（B帧）：是与前一帧和后一帧的逐块差值（帧间编码）



视频连续帧之间的差异往往较小，可将差值进行编码




一组连续的IPB画面
没有I帧，P帧和B帧就无法解码

# 数字音视频与编码

- I帧必须周期性地出现在媒体流中
  - 流媒体直播中，后加入的观众需要收到I帧才能成功解码
  - 如果任意一帧发生了接收错误，则后续非I帧无法解码（由于B/P帧依赖损坏的帧）
  - 快进或者回退到某位置时，解码器需要从该位置前面的I帧开始计算
- 常见帧速率：24帧/秒、30帧/秒、60帧/秒（VR视频）
- 常见分辨率：标清SD 480p/576p、高清HD 720p、全清FHD 1080p、4K超高清 2160p、8K超高清 4320p



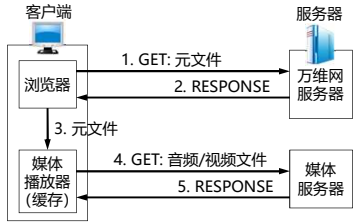






# 2. 流式存储媒体

- 浏览器从服务器下载流媒体文件
  - ✓浏览器用户使用 HTTP 的 GET 报文接入到万维网服务器；这个超链指向一个元文件（有音/视频文件的统一资源定位符 URL）
  - ✓万维网服务器把该元文件装入 HTTP 响应报文的主体，发回给浏览器
  - ✓浏览器调用媒体播放器，把提取出的元文件传输给媒体播放器
  - ✓媒体播放器使用元文件中的 URL，向媒体服务器发送 HTTP 请求报文，要求下载音/视频文件（如对应的某个GOP）
  - ✓媒体服务器发送 HTTP 响应报文，把音/视频文件发送给媒体播放器；媒体播放器边下载边解压缩边播放（通过时间戳同步音频流和视频流）

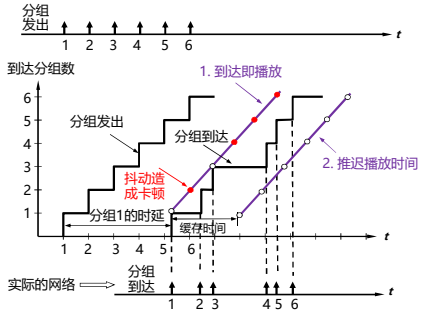



流式存储媒体的典型下载过程



# 流式存储媒体

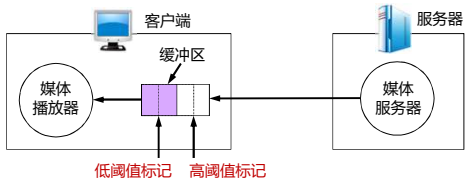
- 发送端以恒定速率产生数据分组
- 网络传输后的结果
  - 由于网络传输的抖动特性，分组到达接收端时变成了非恒定速率
  - 此时如果到达时就随即播放，则会出现卡顿
  - （分组1、3到达接收端可播放，分组2、4、5、6未到达，出现卡顿）
- 如何应对网络传输的抖动特性
  - 在接收端经过缓存后，再以恒定速率播放（推迟播放时间）
  - 能够在一定程度上消除了时延的抖动
  - 但付出的代价是增加了时延






# 流式存储媒体

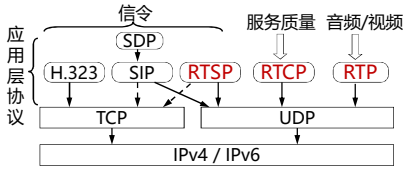
- 客户端缓冲区
  - 客户端播放的是本地缓冲区的内容，而不是立即播放来自网络的实时内容
  - 缓冲区内容小于低阈值标记：数据即将播完，容易出现卡顿；需要加速传输
  - 缓冲区内容大于高阈值标记：增大播放时延，占用存储空间；可以减慢传输
  - 需要的决策：需要多大缓存，服务器以多快速率发送，才能在不稳定的网络中，尽量满足用户期望：高清、低延迟、不卡顿
  - 上述决策需要特定网络协议支持



## 3. 直播与实时音视频

- 实时音频/视频所需要的几种应用协议
  - 一种是信令协议，对建立的连接起控制作用，如RTSP
  - 一种是数据分组传送协议，使音/视频能够以时延敏感属性传送，如RTP/RTCP
- 使用TCP，还是UDP?
  - UDP不可靠但效率高，更适合实时类应用
  - UDP需要自行实现流控算法，增加了成本和复杂性
  - UDP传输音视频可能会被路由器丢弃或防火墙阻拦，而TCP可以畅通无阻

实际流媒体系统往往先尝试UDP，如果失败则转为TCP





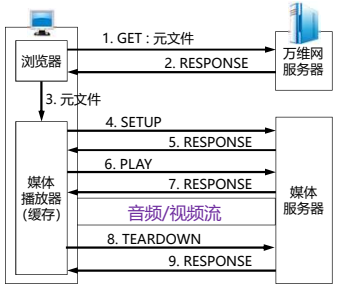

## 直播与实时音视频

- 实时流式协议RTSP (Real-Time Streaming Protocol)
  - RTSP本身并不传送数据，是一个多媒体播放控制协议
  - RTSP对用户下载的实时数据的播放情况进行控制，如：暂停/继续、后退、前进等。又称为“互联网录像机遥控协议”
  - RTSP 是有状态的协议，它记录用户所处于的状态（初始化状态、播放状态或暂停状态）
  - RTSP 控制分组既可在 TCP 上传送，也可在 UDP 上传送
  - RTSP 没有定义音频/视频的压缩方案，也没有规定音频/视频在网络中传送时应如何封装在分组中
  - RTSP 协议本身没有规定音频/视频流在媒体播放器中应如何缓存，由协议的具体实现（和算法）负责



## 直播与实时音视频

- 使用 RTSP 的媒体服务器的工作过程
  - 浏览器向万维网服务器请求音/视频文件
  - 万维网服务器从浏览器发送携带有元文件的响应
  - 浏览器把收到的元文件传输给媒体播放器
  - RTSP 客户与媒体服务器的 RTSP 服务器建立连接
  - RTSP 服务器发送响应 RESPONSE 报文
  - RTSP 客户发送 PLAY 报文，开始下载音/视频文件的特定位置
  - RTSP 服务器发送响应 RESPONSE 报文
  - 开始传输音视频数据（使用RTP协议）
  - RTSP 客户发送 TEARDOWN 报文断开连接
  - RTSP 服务器发送响应 RESPONSE 报文




RTSP 协议的工作过程



## 直播与实时音视频

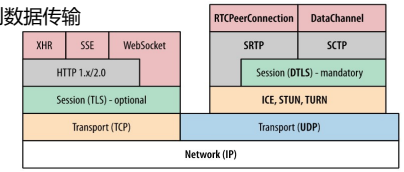
- 实时传输协议 RTP (Real-time Transport Protocol)
  - RTP 为实时应用提供端到端的数据传输，但不提供任何服务质量的保证
  - RTP 是一个协议框架，只包含了实时应用的一些共同的功能
  - RTP 不对多媒体数据块做任何处理，而只是向应用层提供一些附加的信息，让应用层知道应当如何进行处理
- 实时传输控制协议 RTCP (RTP Control Protocol)
  - RTCP 是与 RTP 配合使用的控制协议
  - RTCP 的主要功能：服务质量的监视与反馈、媒体间的同步、播组中成员的标识
  - RTCP 分组也使用 UDP 传送，但 RTCP 并不对声音或视像分组进行封装
  - 可将多个 RTCP 分组封装在一个 UDP 用户数据报中
  - RTCP 分组周期性地在网上传送，它带有发送端和接收端对服务质量的统计信息报告

# 直播与实时音视频

- 网页实时通信 WebRTC（Web Real-Time Communication）
  - 由Google发起的实时音视频通信开源项目
  - 建立浏览器之间点对点的连接，实现音/视频流的传输

开源项目与IETF国际标准同步推进

- WebRTC协议栈
  - 为了满足实时性需求，其核心协议是在右侧基于 UDP 基础上搭建起来的
  - Secure RTP（SRTP）与 Secure RTCP（SRTCP） 是对媒体数据的封装与传输控制协议
  - RTCPeerConnection 用来建立和维护端到端连接，提供高效的音视频流传输
  - RTCDataChannel 用来支持端到端的任意二进制数据传输
  - 流控制传输协议SCTP，提供类似 TCP 的特性
  - DTLS 对传输内容进行加密，是 UDP 版 TLS
  - ICE、STUN、TURN 用于内网穿透，应对NAT等私有地址转换的问题



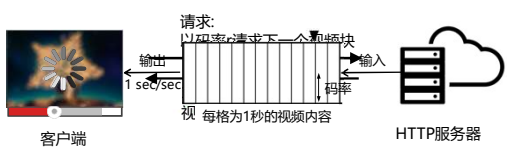


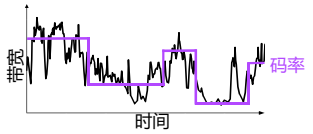

# 4. 流媒体动态自适应传输

- 视频传输优化的挑战
  - 客户端基于当前网络状况，向服务器请求视频块
  - 若视频块的码率>可用带宽：视频块难以及时抵达客户端，出现卡顿
  - 若视频块的码率<可用带宽：视频质量较低，没有充分利用带宽资源
  - 如何为视频块选择贴近当前可用带宽的码率?

视频块的码率 VS 网络可用带宽

- 进一步挑战：网络带宽随时间不断变化




网络可用带宽不断变化时，如何及时选取合适的码率?

引自：Sigcomm'17 Pensieve (MIT)

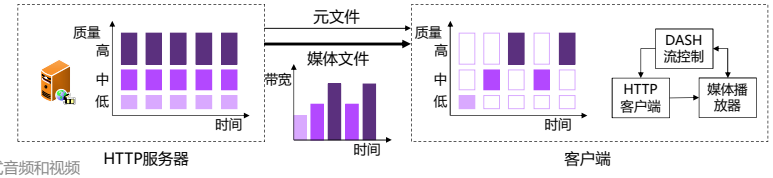


# 流媒体动态自适应传输

- DASH (Dynamic Adaptive Streaming over HTTP)
  - 动态自适应流媒体传输协议DASH，由MPEG组织制定的标准
  - 类似协议：苹果HTTP Live Streaming（HLS）； Adobe的HTTP Dynamic Streaming（HDS）；微软的Microsoft Smooth Streaming

DASH 选取合适码率的协议

- DASH 基本思想
  - 完整视频被拆分为固定时长 (2s-10s) 的视频片段(segment)， 每段提供不同码率
  - 视频片段与其对应的元文件（URL）一同存放于DASH服务器
  - 客户端基于网络条件、缓冲大小等，对每个视频片段，自适应选择合适的视频码率来下载




HTTP服务器 客户端



# 流媒体动态自适应传输

- DASH中普遍使用的自适应码率ABR（Adaptive bitrate）

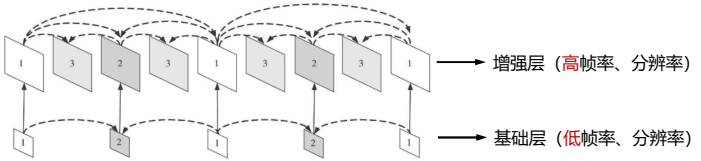
ABR选取合适码率的**算法**

- 2个主要码率切换规则
  - 基于吞吐量的算法：例如：使用滑动窗口平均估计未来吞吐量，选择不高于估计值的最大码率视频
  - 基于缓冲的算法：例如：使用缓冲区满的级别来决定下一个视频块的请求码率
- 放弃请求规则 (Abandon Requests Rule)：在下载视频块的过程中，如果算法检测到下载该视频块的过程有卡顿，则终止当前请求，转而重新下载相应的低码率视频块

- 自适应码率ABR广泛应用于DASH和直播、实时通信等各类流媒体传输
- 由于IBP帧互相不独立，ABR调节往往以GOP为单位，难以实时调节
- 由于网络可用带宽难于预测，因此寻找与可用带宽匹配的最佳码率，很有挑战
- 选高清还是选低清帧的难题：（1）选高清可能卡顿，（2）选低清可能浪费带宽?（3）低清、高清一起传，则收到高清时需要丢弃低清帧，也浪费带宽，如何解决?

# 流媒体动态自适应传输

- 可扩展视频编码 SVC（Scalable Video Coding）
  - SVC 是以H.264为基础，支持多层分级特性：立足基础层，采用锦上添花的增强层
  - 编码器产生的码流包含多个可以单独解码的子码流：不同的码率，帧率和空间分辨率
  - 当带宽不足时，只对基础层的码流进行传输和解码，这时解码的视频质量不高
  - 当带宽充足时，可以传输和解码增强层的码流来提高视频的解码质量
  - 优点：确保传输基础层来避免卡顿，使用富裕的带宽传输增强层，充分利用带宽
  - 缺点：编解码复杂度增加，有多个增强层时开销过大




SVC编码示意图（1、2提升分辨率，3提升帧率）



# 目 录

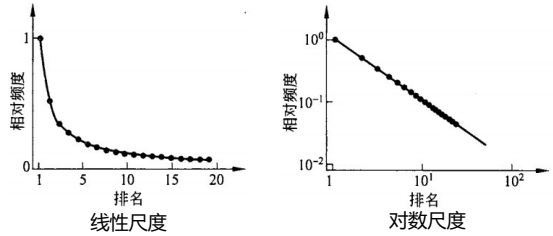

- 7.1 应用层概述（p7）
- 7.2 域名系统（p25）
- 7.3 电子邮件（p74）
- 7.4 WWW（p106）
- 7.5 其它应用层协议（p147）
- 7.6 流式音频和视频（168）
- 7.7 内容分发网络（186）

1. 内容和Internet流量
2. 服务器群和Web代理
3. 内容分发网络CDN
4. P2P网络实现内容分发




# 1. 内容和Internet流量

- 震撼般地转变 (email→FTP→Web→P2P→video)
- 大量站点有很少的流量

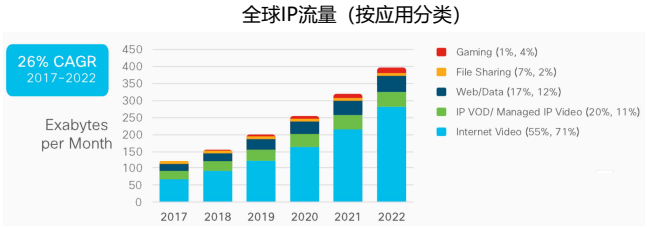



内容分发，尤其是视频，是Internet流量的一个重要构成



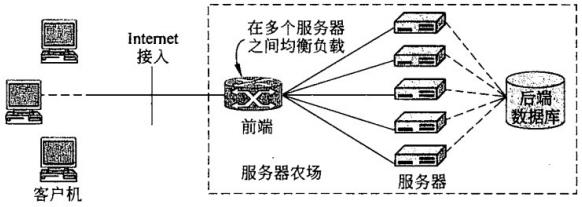
# 内容和Internet流量

- Internet IP流量发展趋势
  - IP Video流量到2022年将占82%



## 2. 服务器群和Web代理

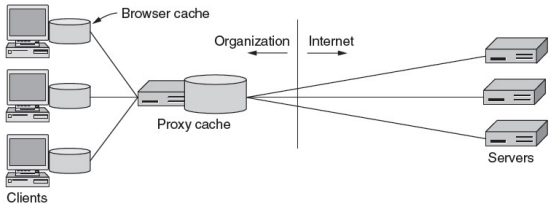
- 服务器群支持大规模Web服务器
  - 前端负责服务器请求的负载均衡
  - 服务器访问相同的后端数据库






## 服务器群和Web代理

- 代理缓存帮助组织扩展Web
  - 将服务器内容缓存到客户上以提高性能
  - 实现组织的策略(如访问策略)

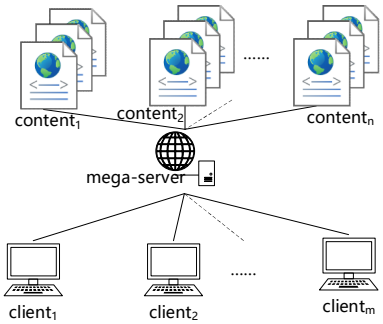





## 3. 内容分发网络CDN

- 问题提出——怎样将内容（如从百万的视频中选定的内容）分发给同时发起访问的数百万用户？
- 单个、大型的“服务器”？
  - 单点故障
  - 网络拥塞
  - 远程用户的长路径

显然，这种解决方案可扩展性差






## 内容分发网络CDN

- 内容分发网络CDN
  - Content Delivery Network，or Content Distribution Network
- 基本思想源于MIT对Web服务瞬间拥塞问题的解决（1998）
  - 一种Web缓存系统，靠近网络边缘（用户）提供内容服务
  - 目前提供更丰富的服务，包括静态内容、流媒体、用户上传视频等
- 主要优点
  - 降低响应时延，避免网络拥塞
  - 避免原始服务器过载及防止DDoS攻击
  - 分布式架构，具有良好的可扩展性
  - 对用户透明，无需用户感知

## 内容分发网络CDN

- CDN服务提供
  - CDN服务提供商：Akamai、蓝讯、世纪互联、网宿等
  - 互联网内容提供商（ICP）：腾讯、百度等
  - 互联网服务提供商（ISP）：移动、联通、电信等
- 关键问题
  - CDN服务器如何布局
  - 在哪里缓存、缓存哪些内容
  - 如何进行重定向，将请求调度到较近或负载较轻的CDN服务器



## 内容分发网络CDN

- CDN服务器布局

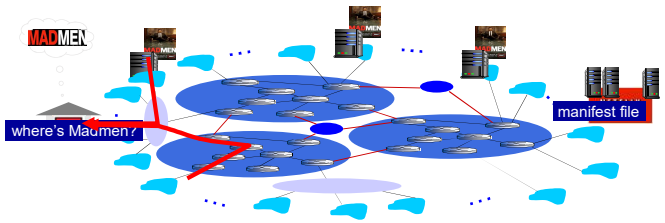
  ——在多个地理分布的位置存储/服务 多个视频拷贝
  - 深入：将CDN服务器深入部署到很多访问网络的内部
    - ✓距离用户要近
    - ✓腾讯云CDN: 2100+节点覆盖国内移动、联通、电信及中小型运营商，及全球50+国家地区，全网带宽120Tbps+
    - ✓Akamai: 240,000部署在 > 120 个国家内 (2015)
  - 邀请做客：在少量(例如10个)靠近接入网的关键位置(例如IXP)，建造大集群，邀请到ISP做客
    - ✓中兴通信ACCESS CDN
    - ✓如Limelight

腾讯云 Akamai ZTE中兴 Limelight NETWORKS



## 内容分发网络CDN

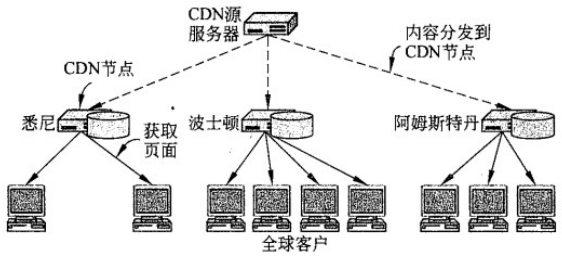
- CDN缓存位置和内容——在CDN节点上存储内容拷贝；如MADMEN
- 订阅者（subscriber）请求内容，服务提供者返回清单CDN
  - 通过清单, 客户端以可支持的最高速率检索到内容
  - 如果网络路径拥塞，可能会选择不同的拷贝或网络速率





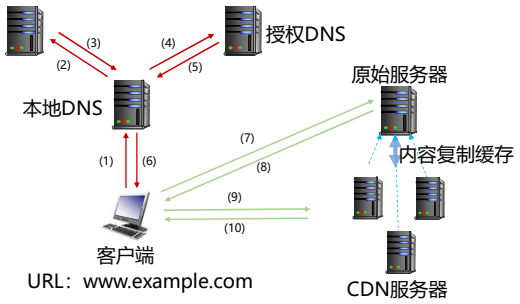

## 内容分发网络CDN

- CDN可以扩展Web 服务器的客户端，这些客户端从就近的CDN结点（或缓存）处得到内容
- CDN分发树



# 内容分发网络CDN

- DNS重定向实现CDN
- 将请求调度到较近或负载较轻的CDN服务器
- HTTP重定向请求内容，服务提供者返回清单CDN
  - ✓ 原始服务器决策CDN服务器
  - ✓ HTTP响应：状态码30X，Location：指明新的位置

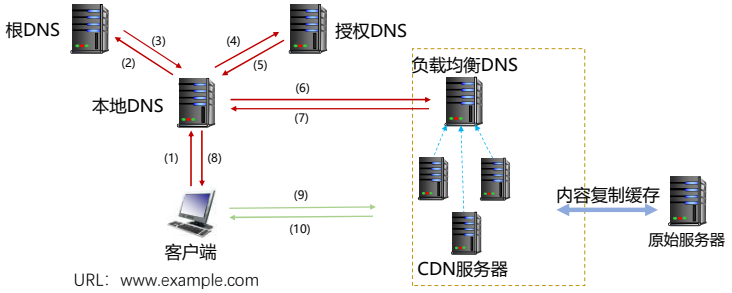



URL：www.example.com



# 内容分发网络CDN

- DNS辅助实现CDN
  - ✓ 负载均衡DNS负责决策CDN服务器选择
  - ✓ 负载均衡DNS需要收集CDN服务器的位置和负载情况
  - ✓ 如果找不到被请求的对象，需要从原始服务器获取




URL：www.example.com



# 内容分发网络CDN

- 网站所有者可以重写通过CDN与内容链接的页面

```
<html>
<head> <title> Fluffy Video </title> </head>
<body>
<h1> Fluffy Video's Product List </h1>
<p> Click below for free samples. </p>

<a href="koalas.mpg"> Koalas Today </a> <br>
<a href="kangaroos.mpg"> Funny Kangaroos </a> <br>
<a href="wombats.mpg"> Nice Wombats </a> <br>
</body>
</html>
```

原始页面

```
<html>
<head> <title> Fluffy Video </title> </head>
<body>
<h1> Fluffy Video's Product List </h1>
<p> Click below for free samples. </p>

<a href="http://www.cdn.com/fluffyvideo/koalas.mpg"> Koalas Today </a> <br>
<a href="http://www.cdn.com/fluffyvideo/kangaroos.mpg"> Funny Kangaroos </a> <br>
<a href="http://www.cdn.com/fluffyvideo/wombats.mpg"> Nice Wombats </a> <br>
</body>
</html>
```

与CDN链接后的某些页面

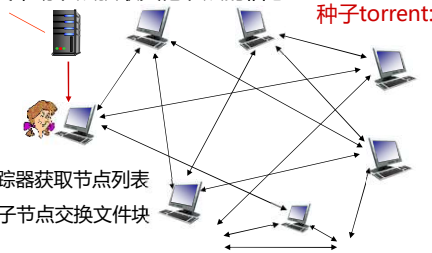


# 4. P2P网络实现内容分发

- P2P文件分发协议：BitTorrent
  - 文件被划分为256Kb大小的块
  - 具有种子(torrents)的节点发送或接收文件

跟踪器tracker: 负责帮助节点获取其他节点的信息

种子torrent: 交换文件块的节点



... 从跟踪器获取节点列表

... 同种子节点交换文件块

# P2P网络实现内容分发

- P2P文件分发协议：BitTorrent
- BitTorrent 使得对等方可以下载种子（torrents）
  - 对等方可以通过Tracker（跟踪器）找到对方，跟踪器是一个服务器，它维护着一个正在主动上传和下载该内容的所有其他对等用户列表
  - 对等节点彼此之间交换各自拥有的块清单，并选择罕见的很难找到的块下载
  - 趋向于匹配那些相互之间具有可比性上传和下载速率的对等节点
  - 一个对等节点为其他对等节点作出的贡献越多，它预期的回报就越大
  - 下载同时向其他节点上传文件块
  - 节点动态加入和退出
  - 获取整个文件后，存在自私离开可能



# P2P网络实现内容分发

- 分布式哈希表（Distributed Hash Tables，DHTs) 是一个完全分布式索引，可扩展到多客户端（或实体）
  - 需要为N个实体，跟踪o(logN)个路径
  - 可以在没有服务器的情况下，使用跟踪器找到对等方
  - 查找DHT中的种子（torrent）来找到对等方的IP地址
  - Kademlia用于BitTorrent，使用了分布式哈希表
- 工作过程
  - 使用一个哈希函数hash把任何IP地址转换成一个160位的数字，该数字称为节点标识符（node identifier），并排好顺序successor函数表示后继节点
  - 有些标识符的节点才对应于实际节点，其余的节点都不存在实际节点
  - 关键字（ key ）也可以由哈希函数产生
  - 节点必须请求successor (hash (torrent))来存储my-IP-address